IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 4 August 2025, accepted 12 August 2025, date of publication 15 August 2025, date of current version 21 August 2025.

*Digital Object Identifier 10.1109/ACCESS.2025.3599190*

SURVEY

# Deep Learning-Centric Task Offloading in IoT-Fog-Cloud Continuum: A State-of-the-Art Review, Open Research Issues, and Future Directions

**GURPREET SINGH CHHABRA[1], GOLUGURI N. V. RAJAREDDY[1,2], ABHIJEET MAHAPATRA[3], S. SUDHEER MANGALAMPALLI[3], (Senior Member, IEEE), KSHIRA SAGAR SAHOO[4], (Senior Member, IEEE), DEEPAK SETHI[3], (Member, IEEE), AND KAUSHIK MISHRA[3], (Senior Member, IEEE)**

[1]Department of Computer Science and Engineering, GITAM Deemed to be University, Visakhapatnam 520045, India
[2]Department of Chemical and Biological Engineering, University of Saskatchewan, Saskatoon, SK S7N 5A9, Canada
[3]Manipal Institute of Technology Bengaluru, Manipal Academy of Higher Education, Manipal 576104, India
[4]Department of Computer Science and Engineering, SRM University-AP, Amaravati, Andhra Pradesh 522240, India

Corresponding author: Kaushik Mishra (kaushik.mishra@manipal.edu)

**ABSTRACT** The rapid growth of IoT and real-time applications has led to a surge in data generation, traditionally processed in cloud-centric architectures. However, this paradigm introduces high latency, bandwidth bottlenecks, and privacy concerns. Fog computing, supported by edge devices, addresses these challenges by enabling computation closer to data sources. This survey presents a comprehensive review of recent studies on task offloading and resource allocation in fog computing, with a focus on Machine Learning (ML) and Deep Learning (DL)-based techniques. We analyze strategies across the fog–cloud continuum, considering factors such as latency, energy consumption, network utilization, and cost. The review also highlights simulation tools, architectural models, and placement policies. Unresolved challenges and interdependencies among research issues are discussed, and future directions are outlined with actionable evaluation metrics. This article serves as a valuable reference for researchers and practitioners aiming to optimize intelligent resource management in fog-enabled IoT environments.

**INDEX TERMS** Fog computing, machine learning, deep learning, Internet of Things (IoT), task scheduling, task offloading, quality-of-service metrics, resource allocation.

## I. INTRODUCTION

Recent decades have witnessed substantial advancements in computing paradigms, with a primary focus on decentralizing computation towards the network edge to minimize user-perceived latency and system overload. The current advancements of IoT and real-time applications in fields like Smart healthcare, Smart gadgets, Smart vehicles, etc. have resulted in an unprecedented rise of Internet-enabled connected devices. Over a decade, these connected devices have significantly increased to over 50 billion, and the exponential growth of data generated from these connected devices has risen to some petabytes [1], [2]. According to the stat, the data generated from IoT devices is likely to grow further by 2030. Traditionally, this data is sent and processed by Cloud Virtual Machines (VMs) in a Cloud-centric IoT model, and the processed outcomes are reverted to the IoT devices. The migration of data from IoT devices to the far-placed Cloud has led to an increase in latency and computational load. Furthermore, the unawareness about the location of the data led to data security and privacy [3].

To address the inherent boundaries of Cloud computing, in 2012, Cisco presented the concept of fog computing, which utilizes the computing nodes in a distributed way to

The associate editor coordinating the review of this manuscript and approving it for publication was Nitin Gupta.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

realize the proximal computation on the edge of the network reducing latency [5]. Since its inception, this paradigm has drawn incredible attention from both academia and the research fraternity to address different challenges faced by the Cloud-centric IoT model. These problems include latency, energy consumption, heterogeneity of tasks & machines, provisioning & de-provisioning of resources, mobility, security, etc. [6]. With the growth of IoT devices and the increasing popularity of Edge computing, fog computing has emerged as a promising paradigm for processing data closer to the edge of the network. In fog computing, tasks are executed on fog nodes, which are geographically distributed to reduce latency and bandwidth usage. Fog computing, though still an evolving paradigm, presents several open research challenges that must be addressed to fully realize its potential. Among these, resource allocation and task scheduling stand out as critical and interdependent sub-problems. Efficient execution of these processes is essential to meet Quality of Service (QoS) requirements, such as low latency, energy efficiency, and reliability, while ensuring optimal system performance. However, achieving effective task scheduling in resource-constrained and heterogeneous fog environments remains a complex challenge due to dynamic workloads, limited computational capacity, and network variability. Given the increasing reliance on fog-enabled IoT systems, there is a pressing need for a comprehensive and up-to-date survey that synthesizes recent advancements and guides future research in this domain.

Building on the critical challenges of resource allocation and task scheduling in fog computing, it is essential to understand its decentralized architecture. Unlike centralized cloud data centers, fog environments consist of geographically distributed and heterogeneous fog nodes deployed closer to end users. These nodes collaboratively process tasks, often requiring task migration to balance workloads and reduce latency.

Users perceive the distributed infrastructure as a unified system, with fog brokers responsible for allocating tasks based on scheduling policies and system status. Traditional approaches include heuristics and metaheuristics; however, ML/DL-based algorithms have emerged as more adaptive and efficient alternatives.

By learning from historical data, ML/DL models can predict optimal task placement considering node availability, bandwidth, processing power, and energy consumption. These intelligent schedulers improve QoS metrics such as latency, energy efficiency, and network utilization, making them well-suited for dynamic and resource-constrained fog environments. This survey specifically focuses on such ML/DL-driven task offloading strategies in fog computing [7], [8], [9].

### A. MOTIVATION

The primary objective of this systematic review is to critically examine and compare ML/DL-based task offloading algorithms specifically within the fog computing environment. Unlike conventional reviews that either broadly address resource management or limit analysis to single paradigms, this survey provides a focused and comparative evaluation of task scheduling techniques applied in the IoT–Fog–Cloud continuum.

Our analysis compares existing surveys across multiple dimensions—including reviewed articles, scheduling methodologies, study duration, simulation tools, computing paradigms, and each survey's strengths and limitations. Many prior works lack a meta-level synthesis, overlooking crucial aspects such as gaps in prior surveys, use of simulation environments, and comprehensive classification frameworks. Furthermore, most concentrate narrowly on load balancing or metaheuristic-based scheduling, without exploring intelligent ML/DL techniques or their adaptability to dynamic fog environments.

In fog-enabled IoT systems, resource contention is common due to constrained and heterogeneous node capabilities. This leads to service disruptions, increased response times, and reliability issues—further magnifying the importance of robust task offloading mechanisms. While resource allocation has been explored extensively in cloud computing, applying such strategies in fog settings introduces new challenges, requiring novel, adaptive solutions.

Resource management in fog computing encompasses multiple operations beyond allocation, including discovery, estimation, monitoring, and provisioning. Existing surveys often adopt a generalist view, lacking granularity in discussing specific components. In contrast, the growing maturity of fog computing now demands focused studies that delve deeper into individual aspects such as ML/DL-enabled resource scheduling.

Notably, very few survey papers emphasize ML/DL-centric offloading tailored to fog computing. This review fills that gap by concentrating solely on intelligent task scheduling across the IoT–Fog–Cloud continuum, offering a detailed meta-analysis, identifying research gaps, and proposing future research directions. The research questions framed in Table 1 are intended to guide the design of efficient, intelligent offloading algorithms and foster a deeper understanding of underlying system constraints and optimization challenges.

### B. NOVELTY

Specifically, this survey is distinguished from prior works in the following ways:

- *Integrated ML/DL-Centric Taxonomy:* We propose a novel taxonomy that categorizes fog computing approaches based on machine learning (ML) and deep learning (DL) techniques applied to task scheduling and resource management. Unlike existing surveys that either focus on traditional heuristics or broadly cover fog architecture, our taxonomy emphasizes intelligent offloading strategies, context-aware learning models, and distributed decision-making frameworks.
- *Comparative Analysis Across Domains:* This survey goes beyond vehicular fog computing by incorporating

**TABLE 1.** Research questions.

| No. | RQs |
|---|---|
| 1. | How can reinforcement learning be leveraged to optimize task scheduling and resource allocation in the IoT-Fog-Cloud continuum? |
| 2. | What role do hybrid DL approaches (e.g., combining CNNs and LSTMs) play in predicting and reducing network congestion in Fog environments? |
| 3. | Can meta-reinforcement learning address dynamic changes in IoT workloads better than traditional RL in fog environments? |
| 4. | How do DL-based anomaly detection models contribute to enhancing resource utilization and failure prediction in the Fog-Cloud continuum? |
| 5. | What are the impacts of task dependencies and deadlines on the efficiency of DL-based task-offloading algorithms? |
| 6. | How can the time complexity of ML/DL algorithms be reduced for resource management in Fog-Cloud data centers? |
| 7. | How can the accuracy of workload prediction be enhanced in ML/DL-based Fog-Cloud task scheduling? |
| 8. | How can training time be minimized when developing ML/DL models for task offloading in the IoT-Fog-Cloud paradigm? |
| 9. | How can energy consumption in Fog-Cloud paradigms be significantly reduced? |

multiple domain-specific case studies (e.g., healthcare, industrial IoT, finance), showcasing how ML/DL techniques generalize across contexts—an aspect underrepresented in existing reviews.

- *Evaluation Metrics Framework:* We introduce a classification matrix that maps each future research direction to quantifiable evaluation metrics (e.g., latency, energy efficiency, cache hit ratio). This actionable framework helps future researchers align design objectives with measurable performance outcomes, something not addressed systematically in the previous surveys (see Table 4/Table 5).
- *Identification of Interdependencies:* In contrast to most surveys that list open issues in isolation, we explicitly analyze the interdependencies among open challenges (e.g., how mobility impacts QoS and offloading). This system-level perspective aids in understanding complex trade-offs and guides multi-objective optimization efforts.

### C. RESEARCH QUESTIONS (RQs)

Important decisions about what type of research to contain, where to get it, how to evaluate its effectiveness, and how to integrate its findings are guided by the review question. It also establishes a certain framework for every review. The review aims to compile the task-offloading aspect of fog computing based on ML/DL algorithms.

The RQs in Table 1 are intended to inspire and assist researchers in addressing some of the major problems associated with ML/DL-centric fog computing. Adhering to the following research questions (RQs) in Table 1. The inferences to these RQs are answered in Section VII.

### D. OUR CONTRIBUTIONS

The contributions of this work can be boiled down as follows:

1. **Comprehensive Survey on ML/DL-Centric Task Offloading:** The manuscript provides a detailed review of ML/DL-based task offloading techniques in the IoT-Fog-Cloud continuum, addressing the gaps in previous surveys by focusing exclusively on fog computing environments and their unique challenges.
2. **Analysis of QoS Metrics:** It highlights various QoS parameters, such as latency, energy consumption, resource utilization, and cost, used to evaluate ML/DL-based resource management algorithms. These metrics are crucial for benchmarking and improving task scheduling efficiency.
3. **Identification of Research Gaps and Challenges:** It outlines the open research issues in ML/DL-based Fog computing systems, including scalability, energy efficiency, workload prediction, task dependency modeling, and simulation tool limitations.
4. **Meta-Analysis of Existing Surveys:** It evaluates prior survey articles in the domain, summarizing their contributions, limitations, and areas of focus, offering a more holistic understanding of the state of the art.
5. **Comparative Analysis of ML/DL techniques for task offloading:** It reviews widely used simulators and programming tools for task offloading and resource allocation in fog computing environments, providing insights into their strengths and limitations.
6. **Proposal of Future Research Directions:** The manuscript suggests potential advancements in ML/DL-based algorithms, such as integrating federated learning, hybrid reinforcement learning, and transfer learning, for improving resource management in fog environments.

### E. SURVEY ROADMAP

Section II provides the research methodology describing the strategy used for searching various research and survey articles along with the process followed for including the final number of papers. Section III outlines the background of ML/DL and fog computing, including the architecture, features, task offloading and scheduling strategies. In Section IV, we present a summary of previous review literature based on ML/DL techniques used in fog computing while the following section V comprises a summary of earlier research work carried out in this domain. The responses to the research inquiries formulated earlier and some future research directions along with the challenges are included in Sections VI and VII correspondingly. Finally, Section VIII offers a succinct synopsis of the work as a conclusion. Figure 1 depicts the flow of the article.

## II. RESEARCH METHODOLOGY

This section provides a detailed methodology of how the research has been carried out in this survey. In addition, different databases and sources from which the relevant survey/review and research papers are taken are delineated afterwards. The existing review/survey articles are systematically reviewed in this area, emphasizing each article's

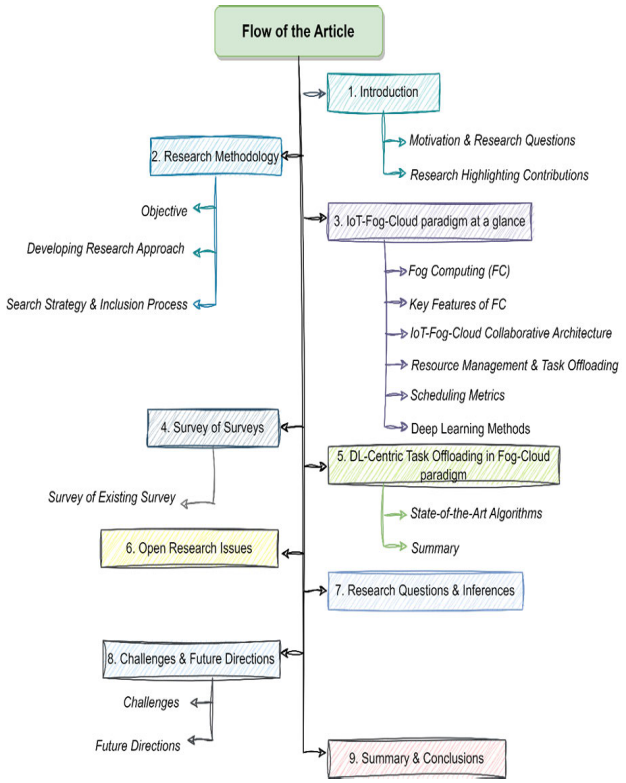



**FIGURE 1.** Review flow.

contributions, the computational paradigm for which the work is conducted, and potential prospects. In addition, it provides extensive coverage of its framework, utilized metrics, and other factors. This review examined articles written between 2010 and 2023 about task offloading and load balancing in emerging paradigms like Cloud and Fog. These include journal articles, conference papers, review and survey articles, and more. Every work has been chosen from scholarly journals that have undergone peer review, comprising Elsevier, IEEE Explorer, IEEE Access, Springer, Wiley, and other international articles.

### A. OBJECTIVE

We meticulously followed the standards in the "Cochrane Handbook for Systematic Reviews of Interventions" [10], the "Preferred Reporting Items for Systematic Reviews and Meta-Analysis: The PRISMA Statement" [11] and the "Systematic Reviews in Educational Research: Methodology, Perspective and Applications" [12] to make sure that this review article is of a high calibre. The following are the key directives for methodically doing a review article:

- Research questions ought to be formulated clearly.
- It is necessary to create a search strategy or research technique.
- The selecting process's criteria must be clearly stated.
- Finding relevant studies and extracting them with the selection process's set criteria, and
- Assessing the research' dependability.

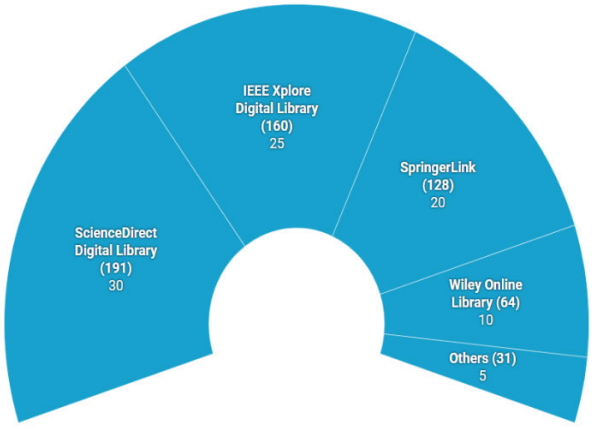



**FIGURE 2.** Research articles considered from various digital library.

- The subsections that follow provide explanations for the guidelines.

### B. FORMULATING A RESEARCH STRATEGY

The comprehensive study that is being offered defines search techniques for finding applicable literary works.

#### 1) LOOKING THROUGH DIFFERENT DATABASES

A large number of conference papers and research articles from highly respected journals that are available online have been assessed. The following list includes the many digital libraries and sources that were investigated for this survey. The comparative chart for the number of articles taken into consideration from different digital libraries is displayed in Figure 2.

- ScienceDirect (https://www.sciencedirect.com),
- SpringerLink (https://link.springer.com),
- IEEE Xplore digital library (https://ieeexplore.ieee.org),
- Wiley Online Library (https://onlinelibrary.wiley.com),
- ACM digital library (https://dl.acm.org),
- Elsevier (https://www.elsevier.com/en-in),
- Multidisciplinary Digital Publishing Institute (MDPI) (https://www.mdpi.com/),
- Taylor & Francis (https://taylorandfrancis.com/journals/),
- John Wiley & Sons Inc. (https://onlinelibrary.wiley.com/),
- Universitatea de Vest (https://www.uvt.ro/ro/),
- SAGE Publications Sage UK (https://uk.sagepub.com/en-gb/eur) and
- Optical Society of America (https://www.osapublishing.org/).

Grey literature [13] sources are:

- ArXiv (https://arxiv.org) and
- Google Scholar (https://scholar.google.com)

#### 2) KEYWORDS FOR SEARCHING

The search term is a string that includes every required keyword and synonym found in the RQs. The variety of terms used to construct the search strings are "Fog computing",

**TABLE 2.** Query string provided into several digital libraries.

| | | | |
|---|---|---|---|
| *Keywords* | "Fog computing" | AND | 'task scheduling' |
| | | | OR |
| | | | 'load balancing' |
| | | | OR |
| | | | 'task offloading' |
| | | | OR |
| | | | 'task orchestration' |
| | "IoT-Edge-Cloud continuum" | AND | 'task scheduling' |
| | | | OR |
| | | | 'load balancing' |
| | | | OR |
| | | | 'task offloading' |
| | | | OR |
| | | | 'task orchestration' |
| | "IoT-Fog-Cloud continuum" | AND | 'task scheduling' |
| | | | OR |
| | | | 'load balancing' |
| | | | OR |
| | | | 'task offloading' |
| | | | OR |
| | | | 'task orchestration' |

''IoT-Fog continuum'', ''IoT-Fog-Cloud continuum'', ''task scheduling'', ''load balancing'', ''task offloading'', ''task orchestration'', and ''machine learning''. These were considered the main keywords. We enclosed our search term within quotation marks ⟨'' ''⟩ to get the exact match of the phrase. Additionally, when creating the search string, we took into account synonyms that were similar to the before specified terms. Scheduling terms include, for example, orchestration, management, processing, and execution. Additionally, the keywords at variance with word formulation (for instance, string including or excluding non-alphanumeric characters like hyphen ⟨−⟩, colon ⟨:⟩, and underscore ⟨_⟩), are taken into consideration. To focus our search, we construct the query string by joining the main terms with their synonyms using the logical Boolean operators AND and OR. The web search engine of numerous online databases is fed with the query string shown in Table 2 and the desired order of the search items is not followed by the results. For instance, ''task scheduling in Fog computing'' and ''Fog computing task scheduling'' gives the similar results.

### C. EXPLORATION

We find many articles from the aforementioned digital libraries from the query string's search results. Though, we did come across several unrelated articles. We reduce the number of comparable papers and omit worthless studies by using advanced search tools provided by several internet databases. The information, which comprises the title and abstract of the publication as well as the keywords and index terms of an article, is considered by advanced search available in every online library.

### D. DEFINITION OF SELECTION CRITERIA

Which papers are eligible to be considered for the survey are determined by the meticulously constructed selection criteria. While unrelated studies are ignored throughout the selection process, studies that are pertinent to the RQs are taken into account. Based on the inclusion and exclusion criteria, irrelevant papers are eliminated after extracting the preliminary results in this systematic literature review.

#### 1) INCLUSION RESTRICTIONS

Given the abundance of research articles in the field of cloud computing, we have considered research articles published from 2014 to 2023. Various articles, particularly explaining the task scheduling and load balancing features in the computing paradigms, published between January 2014 and December 2023 were considered for this survey.

#### 2) EXCLUSIONARY CONDITIONS

Applying the intended criteria stated beneath allows for the elimination of the irrelevant items from the review. The criteria are:

- The articles that do not view the aforementioned computer paradigms as the core of their analysis.
- The papers emphasize workflow scheduling above task or task scheduling.
- Articles that are not full-text accessible and written in languages other than English.
- The results display editorials, prefaces, reviews, summaries of articles, comments, and reader correspondence.

### E. EXTRACTING DATA

Data was taken from the relevant study, and knowledge of the RQs was gathered to comply with the selection criteria. The title of the article, its year of publication, its type (journal or conference), its applications, its problem statement, its problems and future directions are all listed together with the information gleaned from each of the foundational research.

### F. EXAMINE RESULTS

Studies that are found through search are examined to see if they meet the criteria for selection. Through identification, screening, and eligibility, a detailed analysis of the titles, abstracts, keywords, and full texts of the articles is carried out in order to ascertain their relevance to the survey [13]. The logical description of the article evaluation and selection procedure at different stages of the survey is clearly shown in graphic form in Figure 3. The phases of the flowchart are each described in turn.

#### 1) IDENTIFICATION

By using innovative searches on the internet database and grey literature, 513 articles in total were extracted. The articles gathered in this step were forwarded to the following phase's screening procedure.

#### 2) SELECTION AND ELIMINATION

To further address the concern, we emphasize that ''irrelevant articles'' were operationally defined based on the exclusion of studies that (i) did not address task offloading or scheduling

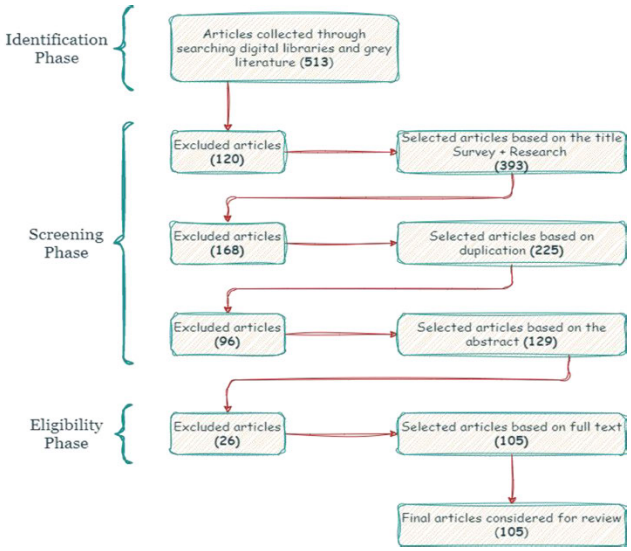



**FIGURE 3.** Filtration of articles over several phases.

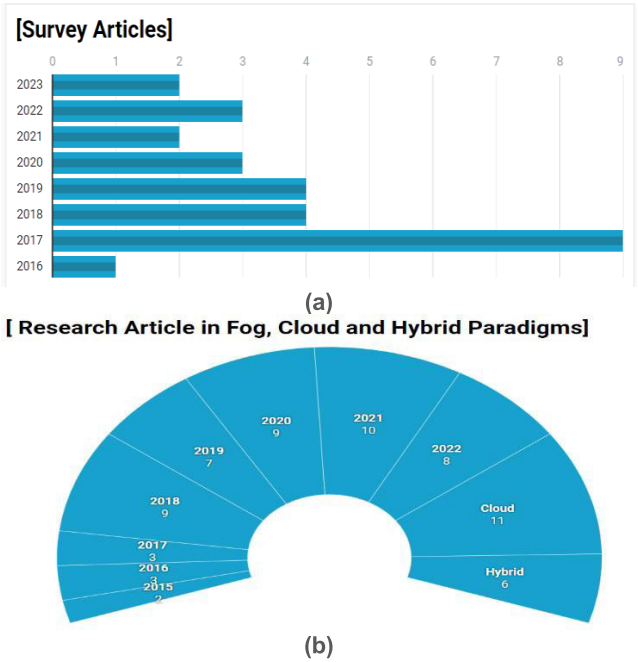



**FIGURE 4.** Articles considered for conducting the survey (a) Survey articles, and (b) Research articles.

in IoT or edge environments, (ii) lacked any ML/DL-based optimization techniques, and (iii) were purely focused on traditional cloud computing without relevance to modern edge-centric paradigms. For instance, articles focusing exclusively on static cloud-based schedulers or those targeting enterprise workload optimization in non-IoT contexts were excluded after title and abstract screening. Some other techniques include as follows:

- Selection based on the title of the article: The selection criteria are established to locate the prominent articles that helped as the baseline for selecting the article titles. Articles around load balancing or task scheduling that have any ambiguity in their titles were excluded from the survey. A total of 393 articles were chosen depending on the title.
- Eliminating redundant articles: Following title-based filtering, the articles that remained relevant were stored in an Excel spreadsheet. The papers were organized by title, publication year, author(s), and place of origin to prevent duplication. This screening led to the discovery of 168 duplicate articles. After the duplicate items were eliminated, the remaining 225 things were chosen for the next phase.
- Selection predicated on the article's abstract: In this case, we studied the abstracts of the articles, looked at the keywords of the chosen papers that made it through the duplicate removal phase, and retrieved the connected publications whose abstracts satisfied the review's objectives. Because of this, we eliminated 96 publications from consideration during this screening phase and chose 129 papers for foundational studies.

#### 3) ELIGIBILITY

Selecting the papers for the systematic literature is an important part of the research. An abstract and a title don't tell you anything about the importance of a work. Article titles and contents don't always correspond. Consequently, studies are chosen for this phase based on the complete text after being assessed based on the title and abstract to determine whether they meet the criteria to be classified as relevant studies. We looked for more pertinent research in each full-text publication's references. Following a meticulous examination of all 129 publications, we ultimately decided to include 105 studies in our analysis.

For the current review paper, Figure 4 presents a study of the number of articles (Survey articles (Fig. 4a), and Research articles (Fig. 4b)) taken into consideration for the fog computing paradigm from 2015 to 2023.

## III. IoT-FOG-CLOUD PARADIGM AT A GLANCE

This section offers preliminaries of the fog computing paradigm, highlighting the core features posed by the fog paradigm, system architecture in collaboration with IoT and cloud, resource management and task offloading, scheduling metrics and machine learning methods.

### A. FOG COMPUTING

Despite its numerous advantages, cloud computing presents several complex challenges, including legal concerns due to cross-border data storage, security and privacy risks arising from lack of control over data placement, and vulnerabilities introduced by virtualization technologies. To address these limitations, Cisco introduced 6 VOLUME XX, 2017 the concept of Fog Computing in 2012, aimed at extending cloud services—such as compute, storage, and networking—closer to the network edge. Analogous to how fog is a cloud

closer to the ground, fog computing bridges the gap between cloud data centers and end devices, offering a decentralized architecture where intermediate fog nodes process and manage data closer to users. This significantly reduces latency, enhances contextual awareness, and improves real-time responsiveness—making it ideal for IoT, smart cities, and latency-sensitive applications. Fog computing is not a replacement but a complement to cloud computing, facilitating seamless integration with Mist, Cloud, and IoT paradigms. It leverages virtualization to dynamically deploy computing and storage resources at the edge, enabling distributed intelligence. By spatially distributing fog nodes between the cloud and IoT devices, it transforms the traditional centralized model into a distributed computing paradigm. While the concept has matured in recent years, aspects such as its architecture, service models, and device requirements remain subjects of ongoing research. In this study, we define fog computing as a decentralized, virtualized computing framework that leverages the processing capabilities of edge devices to optimize task execution and minimize application response times.

Following is a list of the fundamental traits displayed by fog computing [5], [6], [7], [8]:

- Minimal power usage: The communication gap between IoT devices and the geographically distributed fog nodes is reduced, resulting in lower power usage through energy consumption protocols.
- Variability: It facilitates heterogeneity of machines with different specifications, cutting down on waiting times and increasing task completion.
- Hierarchical clustering: The fog layer is divided into hierarchical layers that support various processes depending on the networking layers by hierarchical clustering. The fog nodes are also grouped to maximize each node's computing capacity.
- Rapid Elasticity and Scalability: It is flexible enough to adjust to shifting loads since it can scale up or down in relation to the underlying resources.
- Security and Confidentiality: Advanced security measures and cryptographic principles significantly decrease worries about confidentiality and safety by placing the Fog nodes near the end users.
- Low latency: By moving fog nodes closer to end users, the physical distance between them and IoT devices is greatly reduced, reducing the latency experienced. This feature reduces task offloading over the internet for applications requiring low latency.
- Reduced network bandwidth: Data pre-processing, computation time, and offloading time are shared since the fog paradigm helps information calculation and storage for both the fog nodes and the cloud. Hence, less network bandwidth is required when tasks are completely offloaded to the cloud.

Further, there are some bottlenecks in it, though. Because fog nodes have features like fault tolerance, scalability, security, mobility, interoperability, management of heterogeneous resources, computational issues, and programmability requirements, there are a number of risks involved in implementing them. In this worldview, they are seen as open challenges.

### B. IoT-FOG-CLOUD COLLABORATIVE ARCHITECTURE

Figure 5 illustrates a three-layered architecture of the IoT-Fog-Cloud collaborative computing paradigm that facilitates computations at the edge of the network by leveraging the physical gap and latency overhead of cloud computing. This architecture encompasses three layers of computations such as (1) IoT layer, (2) Edge layer, (3) Fog layer, and (4) Cloud layer. For the sake of understanding and the placement of intermediate nodes, an edge layer is introduced between IoT and fog layers to show the computation at the edge of the network. Every layer is connected through a stable internet connection. An interface (dashed lines between layers) is depicted in Fig. 5 for the seamless migration of tasks among layers through a connection-oriented way.

Each layer is explained as follows: ***(1) IoT layer:*** In the suggested architecture, it is the first and highest layer in the stated architecture. It is the layer closest to the end users, consisting of m number of IoT devices with different configurations, represented as $D = \{D_1, D_2, \ldots, D_m\}, D_j \in D$. They are referred to as smart sensing devices and are spatially dispersed in this layer. Through implanted sensors and actuators, these devices sense the physical data and pass it on to the next higher tier in the hierarchy. These machines are unable to execute such massive tasks with varied constraints, such as priority, deadline, cost, and latency, because of their limited computing and storage capacities. The IoT layer sends these tasks to the cloud or fog in order to achieve these goals.

***(2) Edge layer:*** Edge computing makes it possible for data analysis and processing to take place at the network's edge, near the IoT devices themselves. By performing computations locally, it reduces the latency and bandwidth requirements associated with transmitting data to a central cloud server for processing. Edge devices can run lightweight applications and perform real-time analytics on the collected data. This approach is particularly useful for time-sensitive applications, as it minimizes latency and provides faster response times. In the proposed framework, the edge layer encompasses numerous edge nodes with intermediate nodes for routing the IoT requests either to fog nodes or cloud VMs for processing. The edge nodes can be denoted as $E = \{e_1, e_2, \ldots, e_m\}, e_j \in E$.

***(3) Fog layer:*** This layer uses the physical gap and the service delay during offloading to support the cloud layer. It is composed of $\boldsymbol{m}$ spatially dispersed fog nodes, defined as $\boldsymbol{Fn} = \{\boldsymbol{Fn_1}, \boldsymbol{Fn_2}, \ldots, \boldsymbol{Fn_m}\}, \boldsymbol{Fn_j} \in \boldsymbol{Fn}$ . Fog nodes' decentralised distribution promotes heterogeneity, and their ability to conserve energy gives them an edge over centralised Cloud datacenters. Clusters of heterogeneous and homogeneous computing nodes could be assembled from the fog nodes to improve efficiency. The goal of this configuration of

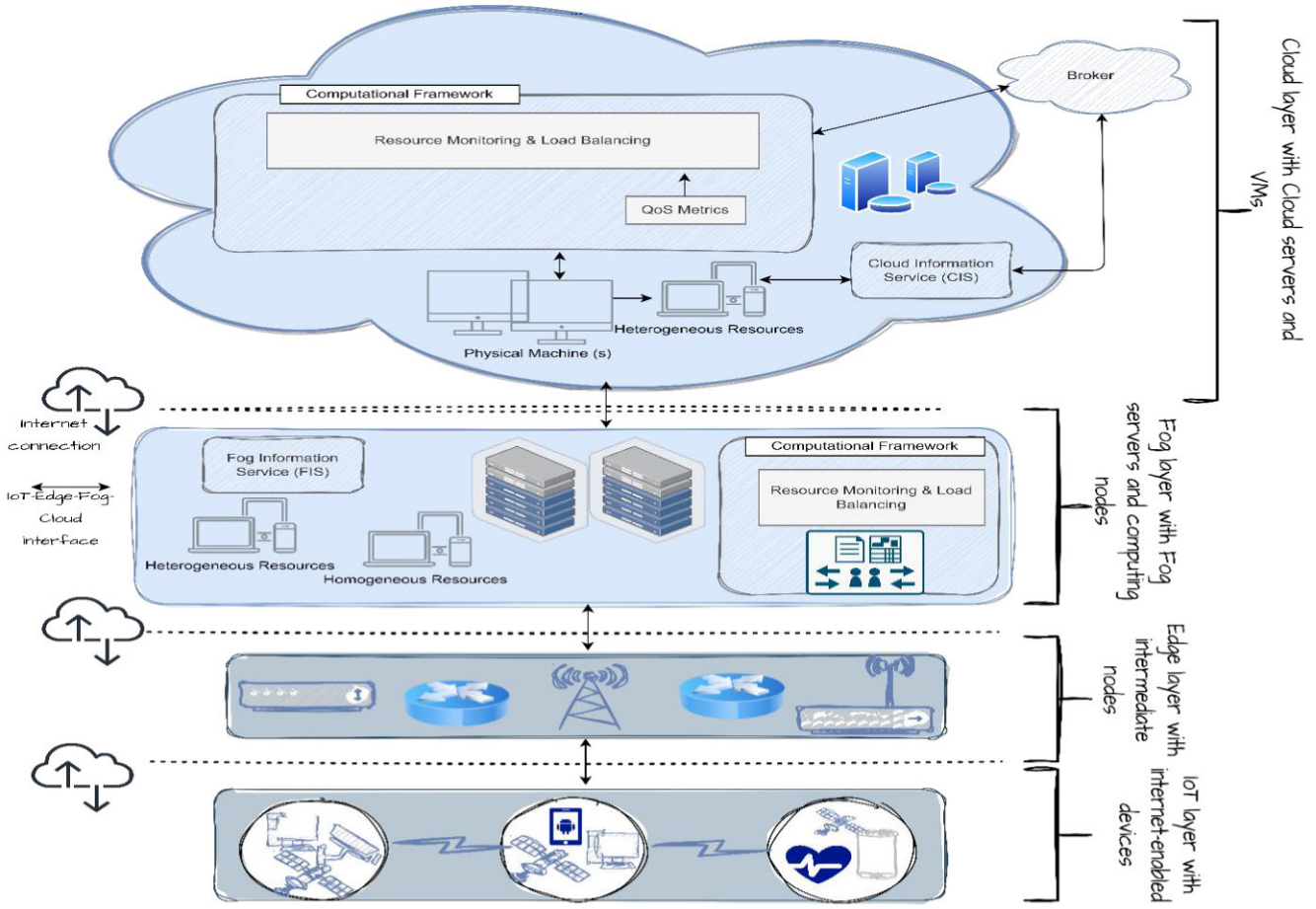



**FIGURE 5.** Collaborative IoT-Fog-cloud architecture.

computing nodes is to manage the ever-increasing demands from end users in a dynamic manner. In addition, the fog nodes are associated with the cloud servers so that work can be offloaded in the event of any unfavorable events, including fog server crashes or fog node malfunctions. By adding further levels to this fog layer, a hierarchical fog layer with n levels fulfilling different purposes might be formed. In this context, fog nodes are regarded as physical or virtual components in charge of processing tasks with various requirements. Fog computing builds upon the concept of edge computing by adding a layer of intermediate computing and data storage. It involves deploying more powerful computing resources, such as edge servers or gateways, at the edge of the network.

Although fog computing and edge computing are frequently misunderstood, they differ greatly from one another. Fog computing, which runs applications in a multitier architecture, blends and separates the hardware and software functions, enabling dynamic reconfiguration for different applications even when they are carrying out intelligent computing or transmission services. On the other hand, edge computing directly provides transmission service by executing applications in a fixed location. The fog environment is indeed very hierarchical, and edge computing is often limited to a small number of endpoint appliances and devices. In addition, the fog layer handles data processing, control, and storage acceleration in addition to networking and computation.

**(4) Cloud layer:** Cloud computing represents the centralized infrastructure, typically hosted in remote datacenters, where vast computational resources and storage are available. The cloud serves as a hub for data storage, advanced analytics, machine learning, and long-term data archiving. It provides scalability, high-performance computing, and access to a wide range of services and resources. Cloud computing is ideal for handling massive data volumes, running complex algorithms, and performing in-depth analytics that require substantial computational power. Several servers that allows for on-request resource allocation and services makes up the primary cloud datacenter. This layer is made up of a variety of physical machines, designated as $\boldsymbol{P} = \{\boldsymbol{P_1}, \boldsymbol{P_2}, \ldots, \boldsymbol{P_m}\}, \boldsymbol{P_j} \in \boldsymbol{P}$, each with a particular set of requirements. Additionally, a virtualized environment is built to enable the modelling and modelling of cloud entities by constructing a $\boldsymbol{m}$ number

of virtual machines (VMs) out of the physical machines $VM = \{VM_1, VM_2, \ldots, VM_m, VM_{j(1,2,\ldots,m)} \in P\}$. VMs are regarded as nodes with high storage requirements and efficient computing. As a result, it manages high-end assignments with strict timeframes. However, because it is a centralised datacenter, it cannot manage massive tasks with competing limitations brought on by IoT devices. By moving the calculations from the cloud VMs to fog nodes, the fog layer assumes the workload and ensures productivity, to take advantage of this intrinsic constraint. As a result, the cloud layer and the fog layer are complementary. Furthermore, storage is done via the cloud layer.

The IoT–Fog–Cloud collaborative architecture, as depicted in Figure 5, is composed of four hierarchical layers: (1) the IoT/Edge layer, (2) the Fog layer, (3) the Core network layer, and (4) the Cloud layer. Each layer participates in distributed task processing and data analytics, but efficient system performance depends heavily on intelligent task offloading decisions between these layers.

The offloading decision boundaries between each layer are presented as follows:

- Edge to Fog Offloading: Tasks are offloaded from IoT devices to the Fog layer when they exceed the device's processing or energy capabilities but require low latency (e.g., <100 ms). Typical examples include real-time object detection in surveillance or local anomaly detection in healthcare devices.
- Fog to Cloud Offloading: When tasks involve high computational demand (e.g., deep learning inference), big data analytics, or long-term storage requirements, and are not latency-sensitive, they are offloaded to the cloud. Examples include historical trend analysis, model retraining, and batch processing.
- Edge to Cloud Direct Offloading: In scenarios where the fog layer is congested or unreachable, and the task can tolerate delay (e.g., periodic backups or environmental data aggregation), direct offloading to the cloud may occur.

The ML/DL-driven decision logic for task offloading in fog paradigm is presented as follows:

- Workload prediction models (e.g., LSTM, ARIMA) are integrated at the Fog layer to forecast peak periods and proactively offload tasks to the cloud or balance them across nodes.
- Reinforcement learning agents at the Edge/Fog layers dynamically learn optimal offloading strategies based on contextual factors such as CPU utilization, delay constraints, bandwidth availability, and user preferences.
- Classification models (e.g., SVM, decision trees) can be used at the edge to categorize tasks into low-latency critical, latency-tolerant, or compute-intensive classes, determining the most appropriate layer for execution.

This dynamic, ML/DL-informed decision-making mechanism ensures adaptive, context-aware task distribution, optimizing both system performance and QoS requirements. Overall, this continuum allows for a balanced approach to IoT system design, taking advantage of the strengths of each layer and optimizing the utilization of network resources.

### C. RESOURCE MANAGEMENT AND TASK OFFLOADING IN FOG-CLOUD PARADIGM

Resource management is essential due to its focus on optimizing the utilization of already available resources. The steps in this procedure are designed to be simple and practical while using a modest amount of computer resources. In contrast to cloud computing, which, from the perspective of a single user, gives the impression that resources are limitless, the resource management process is heavily explored in fog computing since computational resources are frequently constrained and must be used efficiently.

In fog computing, the lifespan of a load balancer is represented as a structure. After evaluation, the user's appeal is sent to the appropriate datacenter built on the available resources. However, care should be taken to ensure that servers (VMs) are dispersed evenly and are neither overloaded nor underloaded. Because of this, the idea of load balancing is introduced. A reliable load balancer is required to track how well fog apps perform. Task Scheduling is one of the many factors that contribute to insufficient load distribution. At the fog's back end, load balancing typically takes place.

In a collaborative IoT-Fog-Cloud paradigm, task scheduling and task offloading are crucial aspects of optimizing resource utilization and improving overall system performance. Task scheduling involves determining which tasks should be executed and allocating resources to complete those tasks efficiently. Task offloading, on the other hand, refers to the process of transferring tasks from one computing entity (e.g., an IoT device or fog node) to another (e.g., a fog node or cloud server) for execution.

In the context of fog computing, here are some key considerations and techniques related to task scheduling and task offloading:

*1. Task Characteristics:* Different tasks have varying requirements in terms of computational resources, response time, and data dependencies. Task scheduling algorithms should consider these characteristics to make optimal decisions. For example, tasks with strict real-time requirements might be scheduled on nearby fog nodes to minimize latency, while computationally intensive tasks may be offloaded to more powerful fog nodes or the cloud.

*2. Resource Availability:* The availability and capacity of computing resources within the fog computing environment should be considered during task scheduling. This includes factors such as the processing power, memory, storage, and network bandwidth of fog nodes. Scheduling algorithms need to evaluate the current resource utilization and select the most suitable node to execute a task.

*3. Load Balancing:* To ensure efficient resource utilization, load balancing techniques can be employed. These techniques distribute tasks across fog nodes in a way that

balances the workload and minimizes resource bottlenecks. Load balancing algorithms can consider factors such as the current load on each node, available resources, and network conditions to make intelligent decisions.

*4. Energy Efficiency:* Energy consumption is a critical concern in IoT and fog computing environments. Task offloading can be used strategically to minimize energy consumption by leveraging the computational resources of more powerful nodes or clouds. Offloading decisions should consider the energy efficiency of different nodes and prioritize offloading tasks to nodes that consume less energy for a given workload.

*5. Context Awareness:* Task scheduling and offloading can be enhanced by incorporating context awareness. This involves considering contextual information, such as location, network conditions, device capabilities, and user preferences. Context-aware algorithms can make informed decisions about task assignments and offloading based on the current context, optimizing performance and user experience.

*6. QoS Requirements:* QoS requirements, such as response time, reliability, and data security, need to be considered during task scheduling and offloading. Scheduling algorithms should prioritize meeting these requirements and allocating resources accordingly. For example, latency-sensitive tasks can be assigned to nearby Fog nodes or processed locally to ensure low response times.

*7. Decision-Making Strategies:* Various decision-making strategies can be employed for task offloading. These strategies can be rule-based, heuristic-based, or even leverage machine learning techniques to make intelligent offloading decisions based on historical data, task characteristics, and system conditions.

The goal of task scheduling and task offloading in a collaborative IoT-Fog-Cloud paradigm is to optimize system performance, minimize response times, reduce energy consumption, and effectively utilize the available computing resources. By considering task characteristics, resource availability, load balancing, energy efficiency, context awareness, and QoS requirements, organizations can design effective scheduling and offloading mechanisms to enhance the overall performance of the system.

### D. SCHEDULING METRICS

This section is a list of the criteria that most researchers use to evaluate the present state of the art. These factors are crucial for creating and implementing a load-balancing algorithm. These variables regulate the algorithm's performance regarding the fog application. Researchers use some of these characteristics to validate the suggested methodology, and they should be modified to prevent problems in the fog computing environment. The authors divided the metrics used in the available literature into qualitative and quantitative categories, as shown in the hierarchy map. Control factors based on recent studies include getting a current version and improving the accuracy of the representation. Nine separate formative and summative assessments were used in this research study's analysis of recent literature, as indicated and demonstrated below:

*1. Resource Utilisation (RU):* The quantity of system resources, such as memory and CPU, that are in use. A fog datacenter's RU level is being evaluated as the primary objective goal. Due to the increase in demand for services, resource utilization is required. For the load balancing algorithm to work at its peak, significant resource utilisation is needed.

*2. Scalability* ($S$) : Like a system, an algorithm must be capable of performing effectively under unanticipated circumstances. In contrast, when workload and task count rise, the algorithms ought to be modular. The load balancing method needs to be highly adaptable to work well.

*3. Throughput (TP):* How many task proposals have been handled and completed successfully in the VM in each amount of time determines it. The amount of data that is transferred from one location to another is implied by throughput. For the load-balancing system to work properly, a high TP is necessary.

*4. Response Time (RT):* Response Time (RT) is how long the algorithm would take to respond to a task. We should consider the time spent commuting, serving, and waiting when calculating it. It's the amount of time it takes to answer a user's question. A load-balancing algorithm must have a low RT to be effective.

*5. Makespan (MS):* The overall completion time is crucial for completing all tasks and giving users of the system resources. It is an essential prerequisite for the fog computing environment's scheduling stage. It calculates how long it takes to finish a collection of tasks. The MS of a reliable load-balancing algorithm would be low.

*6. Associated Overhead (AO):* Associated overhead is the amount of delay encountered throughout the implementation of the load balancing algorithm. The high number of task migrations and inter-process communications could be the cause of it. The load-balancing approach has a small amount of overhead when managing the system's load.

*7. Service Cost (SC):* the cost associated with the services being offered and executed in a computing layer. This cost is established during the negotiation between the consumer and provider in SLA. If the provider fails to provide certain services, then it must pay a penalty to the consumer for the delay in service.

*8. Service Delay (SD):* the delay caused by the execution of the services in a certain computing layer. It accommodates the processing delay, communication delay and response delay.

*9. Fault Tolerance (FT):* The load balancing system should function effectively even if some components fail. In a different sense, another VM should carry out the tasks if one VM is overloaded. FT directly affects how well the load-balancing algorithm performs. The efficiency of the load balancing method will also be good if the value of FT is high.

*10. Migration time (MT):* The whole amount of time needed to migrate a task from one virtual machine to another is called migration time. The migration process needs to start earnestly without affecting the system's functionality. It is

heavily reliant on the virtualization strategy used by fog. The value of migration should be below for effective load-balancing performance.

*11. SLA Breach:* Prioritise the parts of SLA violations that have been decreased based on deadline restrictions. When resources (VMs) become unusable due to overload, SLA breaches occur. For a greater level of customer perception, a minimum SLA is advised.

### E. MACHINE LEARNING (ML) OR DEEP LEARNING (DL) METHODS IN TASK OFFLOADING

Task offloading is an NP-hard problem in computing paradigms like cloud, edge, fog, etc., because of the randomness of activities due to their varied quantities, durations, priority, arrival times, and deadlines, as well as timeliness consideration. The scheduling of random tasks requires high-speed, computationally efficient algorithms due to these contradicting but demanding properties. It appears that meta-heuristics cannot be very effective at dealing with task randomization and completely adjusting the dynamic computing environment, unless they are modified for the problem [102].

Using statistical techniques, DL [101] is a cutting-edge methodology that may be able to forecast the unpredictable nature of randomly assigned tasks in constantly changing computer environments and enhance performance. This term comes from the general term ''Deep Learning'' (DL), which is an artificial neural network (ANN) based system. An intriguing technique for handling a variety of artificial intelligence (AI)-related applications is machine learning. DL techniques can automatically learn from the raw data, lessen the effort required for feature extraction, and enhance overall prediction performance. ML approaches can be broadly categorized into four groups depending on the type of data used in the learning method: supervised, semi-supervised, unsupervised, and reinforcement learning.

- *Supervised Learning [14]:* One of the common ML/DL algorithm categories relies on supervision (previous knowlEdge). With this method, inputs, outputs, and some properties are included in the ''labelled'' or ''known'' datasets used to train the algorithm. The training dataset refers to these data sets. The machine generates a model based on training that accurately forecasts the patterns or properties of labels. Even if the label-feature pairs have already been entered into the model as input, it nevertheless uncovers new patterns in the relationship between the labels and the features. Numerous real-world issues can be addressed using it, including fraud detection, risk assessment, spam detection, etc. The two main tools in this area are regression and classification. Nevertheless, this method needs more computing time to train, especially for complex tasks.
- *Unsupervised Learning [15]:* It doesn't utilise previous experience, in contrast to supervised learning. In this case, characteristics rather than labels are used to train the algorithm. The main goal of this technique is to find hidden connections and structures in the unlabeled data by clustering or categorizing it according to the input attributes. It classifies the unlabeled data more precisely. Ultimately, it discovers a better method to group these underlying structures into clusters that seem more organised. As it adjusts to the dynamically changing inputs, this model's decision-making capabilities gradually improve and become more refined. When handling more complicated problems, this learning approach is more appropriate than supervised learning. Nevertheless, if the data is not previously tagged or the model is not trained, the result may be less accurate. Anomaly detection, recommendation systems, and other specialised problem fields are only a few examples of the numerous uses it has.
- *Semi-supervised Learning [16]:* This method blends both supervised and unsupervised learning approaches. This method is used to overcome each other's weaknesses by utilising their respective strengths. An unlabelled dataset, for instance, can benefit from some extra features when solving classification issues. Meanwhile, the labelled data that contains simple features may make clustering more effective. It is effective but has a low accuracy rate.
- Reinforcement Learning [17]: It takes its cues from how people absorb knowledge from their experiences. Unlike supervised learning, there is no tagged data present here. Through ongoing trial-and-error interactions with the environment, a software component, and an agent, balance exploration and exploitation. Here, a reward system and an interpreter are used to facilitate learning. Every time the algorithm loops, the agent detects the output and provides it to the interpreter. If the output is positive, the interpreter then interprets the message. If the result is positive, the agent is rewarded or reinforced. If not, the algorithm keeps trying until it comes up with an acceptable solution. Often, the incentive structure is limited to the treatment's effectiveness. In new computer paradigms, this technique is often used in conjunction with deep learning for resource management. In order to reduce waiting time for tasks, it is used to plan the resources' availability.

### F. PRACTICAL APPLICABILITY OF ML/DL MODELS IN FOG COMPUTING

While various machine learning paradigms—supervised, unsupervised, semi-supervised, and reinforcement learning—offer unique strengths, their practical applicability in fog computing environments depends significantly on scenario-specific constraints such as latency sensitivity, energy availability, model size, and computation resources at the edge.

- In low-latency and resource-constrained environments (e.g., real-time healthcare monitoring or industrial automation), lightweight models like decision trees,

**TABLE 3.** Decision framework for model selection.

| Scenario | Resource Constraint | Model Preference | Justification |
|---|---|---|---|
| Real-time control (low-latency) | High | QNNs, Logistic Regression | Fast inference, low power |
| Video analytics | Moderate to Low | CNNs, RNNs | High accuracy needed for frame-by-frame tasks |
| Dynamic task offloading | Varies | RL, DQN | Learns optimal policy under uncertainty |
| Anomaly detection (IoT security) | Moderate | Autoencoders, Hybrid models | Balance between generalization and detection |

shallow neural networks, or quantized neural networks (QNNs) should be prioritized. These models can operate within limited memory and processing capabilities while ensuring acceptable accuracy.

- In contrast, high-accuracy scenarios such as video analytics or anomaly detection in critical infrastructure may benefit from deep convolutional or recurrent neural networks, provided the fog nodes are equipped with hardware accelerators or collaborate with the cloud tier.
- For dynamic environments with sequential decision-making needs (e.g., vehicular fog networks), reinforcement learning (RL) and deep RL can adapt task offloading policies over time. However, their high training overhead suggests offloading the model training to the cloud while deploying pre-trained, lightweight agents at the fog layer.

Table 3 presents a decision framework for model selection which enables context-aware deployment of intelligent offloading solutions, improving scalability, efficiency, and QoS in fog-based systems.

### G. CASESTUDIES ON THE ROLE OF ML/DL IN TASK OFFLOADING

1. *Vehicular fog Computing (VFC):* To manage the deluge of data, the city is equipped with a VFC infrastructure. Roadside fog nodes, such as smart traffic lights or dedicated Fog servers installed at intersections, act as intermediaries between the vehicles and a central cloud. These fog nodes process data locally to reduce latency and bandwidth usage.
2. *Deep Learning for Real-Time Video Analysis:* Vehicles upload video streams from their onboard cameras to nearby fog nodes. Using convolutional neural networks (CNNs), the Fog nodes analyze these streams to detect traffic violations, predict congestion hotspots, or identify accidents. This offloading reduces the computational burden on vehicles and ensures real-time analysis.
3. *Reinforcement Learning for Dynamic Task Offloading:* A reinforcement learning (RL) agent deployed in the VFC system continuously monitors resource availability (e.g., CPU, memory, and bandwidth) of the fog nodes and the urgency of tasks. For example:
   - Low-priority tasks, such as uploading environmental data, are offloaded to remote cloud servers.
   - High-priority tasks, such as accident detection, are processed locally at the fog node closest to the event. This dynamic decision-making minimizes delays and optimizes resource utilization.
4. *Federated Learning for Collaborative Model Training:* Vehicles and fog nodes collaboratively train models using federated learning (FL). For instance, data collected by multiple vehicles is used to improve a deep learning model for collision prediction [105] without sharing raw data. Only model updates are exchanged, preserving privacy and reducing communication overhead.
5. *Load Prediction with Machine Learning:* Predictive ML models analyze historical traffic data and current conditions to anticipate workload spikes (e.g., during rush hours or major events). The VFC system preemptively offloads tasks to alternative Fog nodes or the cloud, avoiding congestion in the network.
6. *Smart Healthcare:* Privacy-Preserving Patient Data Offloading: In smart healthcare systems, wearable devices and mobile health monitors continuously collect sensitive physiological data. Offloading these data to cloud or fog servers for processing introduces severe privacy concerns. Federated Learning (FL) enables distributed model training across multiple hospitals or user devices without transmitting raw data. For example, FL has been used for predicting patient deterioration in ICUs while ensuring compliance with data privacy regulations like HIPAA. By processing encrypted model updates rather than patient records, FL allows fog nodes to personalize care while preserving privacy, thereby making fog computing a viable platform for real-time health analytics.
7. *Industrial IoT:* Overcoming the "Data Silo" Problem. In Industrial IoT (IIoT) environments, machines, control systems, and factory floors generate heterogeneous, high-volume data that are often isolated within organizational boundaries—creating data silos. Federated learning and multi-agent reinforcement learning can enable collaborative predictive maintenance across different plants without exposing proprietary information. By sharing only model weights, multiple factories can collaboratively train fault detection or energy optimization models at fog layers, improving system-wide intelligence without risking data leakage or IP theft.

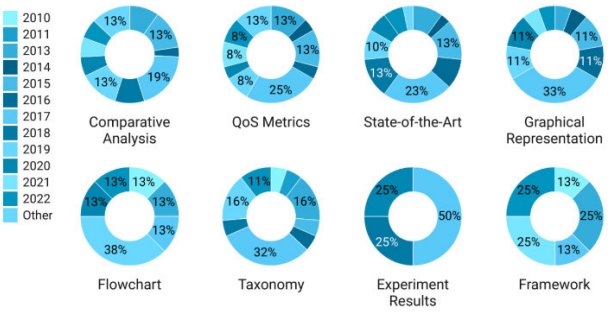



**FIGURE 6.** A comparison of current surveys.

8. *Financial Fog Systems:* Credit Scoring in Decentralized Networks. In the financial sector, tasks such as credit risk evaluation and fraud detection depend heavily on user behavior data, which are often spread across banks, retailers, and telecom providers. A fog-enabled system using privacy-preserving ML, such as homomorphic encryption combined with FL, can securely perform distributed learning across these entities. This supports real-time fraud detection and personalized credit scoring while maintaining user confidentiality. Such approaches have proven effective in reducing task response times and improving trust in mobile banking fog systems.

This scene highlights the critical role of ML/DL in enhancing the efficiency, responsiveness, and scalability of task offloading in vehicular fog computing, making it a cornerstone for intelligent transportation systems.

## IV. SURVEY OF EXISTING SURVEYS

During these years, tiny study has been published on the most current state-of-the-art load balancing and task offloading methods for the Fog-Cloud field. Nevertheless, none of these review studies go into considerable length in comparing the previous literature with the existing survey. A comprehensive analysis is crucial because it sheds light on the numerous load-balancing and task-offloading tactics used in these emerging paradigms. Numerous survey articles solely discuss one load-balancing model or task-offloading model. However, this current review suggests cutting-edge load-balancing and task-offloading methods in fog and cloud. An alternative contributing aspect is the attention and assessment of the utilized performance constraints, which is a drawback of the earlier review work. This segment makes two contributions. In the first place, it presents a compilation of the previously published review papers that have already been written in terms of contributions, computing paradigm, and potential coming directions, as shown in Table 4. Next, it offers a comparison of current review papers under the subsequent restrictions, as shown in Table 5, including a comparative examination performed, graphical representation, flowchart, taxonomy, recommended outline, etc. Figure 6 compares the evaluation of the current survey items from 2010 to 2023 regarding the constraints mentioned.

## V. SDL-CENTRIC TASK OFFLOADING IN FOG-CLOUD PARADIGM

It is an NP-hard problem to schedule in computer paradigms such as cloud, edge, fog, etc. since tasks are unpredictable in nature due to their shifting quantities, durations, priorities, arrival times, and deadlines. These contradictories yet complex traits necessitate high-speed, computationally effective algorithms for work schedules. To totally alter the dynamic computing environment and deal with the randomness of workloads, traditional metaheuristics appear ineffective if they are not modified for the problem [103]. Modern approaches like ML or DL can predict the unpredictability of randomly generated tasks in constantly changing computer environments and boost performance. ML/DL appears to be an intriguing technique to solve numerous applications linked to AI. ML/DL methods can reduce feature extraction effort, improve performance (prediction), and automatically understand raw data. ML or DL techniques can be roughly classified as Supervised, Semi-supervised, Unsupervised, or Reinforcement learning, depending on the type of data utilized in the learning process.

In order to address the scheduling problem, we have examined a few DL-centric techniques applied in computing paradigms. We have outlined the key points in terms of the objectives of the study, the test cases used, the types of tasks and servers used, the compared algorithms, the different computing paradigms, the advantages of every method, and the direction that the research will take in the future. In Table 6, this summary is displayed.

In order to address the problem of task scheduling in a cloud computing context, the paper [70] proposes a novel way of utilizing deep Q-learning. First, the challenges of task scheduling are examined in the context of Cloud computing, taking into account the dynamic nature of the workload and the need to maximize resource use. The idea of Q-learning and deep Q-learning is then introduced, explaining how they might be utilized to address the scheduling issue. A training phase and a scheduling phase make up the proposed scheduling plan. The deep Q-learning method is used in the training phase to determine the best scheduling strategy based on training data. The scheduling phase then makes real-time scheduling decisions using the learned policy.

In the article [72], it is suggested a novel method for employing deep reinforcement learning to solve the task scheduling issue in a cloud manufacturing context. The study's first portion discusses the challenges associated with task scheduling in a cloud manufacturing environment, including maximising resource usage, cutting processing times, and ensuring high-quality results. The proposed scheduling approach consists of three phases: a training phase, a testing phase, and a scheduling phase. In the training phase, the deep reinforcement learning algorithm is used to learn the optimal scheduling policy based on training data. In the testing phase, the available policy is evaluated on a separate set of data to assess its effectiveness. Finally, the learned

procedure is used to make real-time scheduling decisions in the scheduling phase.

The study [73] proposes a novel method for optimizing deep reinforcement learning-based task offloading strategy in mobile edge computing. The significance of work offloading in mobile edge computing and the demand for an effective offloading strategy are covered in the paper. It then introduces the concept of deep reinforcement learning and explains how it can optimize the offloading process. Three phases make up the suggested offloading strategy: a training phase, a testing phase, and an offloading phase. The deep reinforcement learning method is utilized in the training phase to determine the best offloading strategy using training data. The learned policy is tested on a different data set at this step to determine its efficacy. Finally, during the offloading phase, judgments on offloading are made in real time using the learned policy.

A multi-agent deep reinforcement learning (MADRL) method is suggested in the paper [74] for simultaneous multi-channel access and task offloading in mobile-edge computing (MEC) networks, intending to optimize the performance of Industry 4.0 applications. The paper begins by introducing the concept of Industry 4.0 and its challenges in data processing and communication. The authors then describe MEC as a promising solution to these challenges, potentially enabling low-latency and high-bandwidth data processing at the network edge. The suggested MADRL strategy is made to dynamically distribute resources among numerous channels and choose the best task-offloading option for each user in a MEC network.

In a blockchain-enabled Internet of Things (IoT) network with edge computing, a deep reinforcement learning (DRL) technique is provided by the paper [75] to optimize resource allocation for delay-tolerant data. The article starts off by outlining the difficulties that IoT networks face, such as a lack of resources and the requirement to process and store a lot of data. The proposed DRL approach is made to maximise resource allocation in an edge-enabled IoT network with blockchain technology while considering the data's tolerance for delay. The method trains the agents using a deep Q-learning algorithm with experience replay, enabling them to gain knowledge from the past and develop their decision-making skills over time.

For mobile Edge computing (MEC) networks, the study [76] suggests a strategy for intelligent resource allocation that depends on federated learning (FL) and deep reinforcement learning (DRL). The problems that MEC networks must overcome to allocate their limited resources effectively are discussed at the outset of the paper. The authors go on to explain how DRL and FL can help with these issues by enabling intelligent resource allocation and enhancing MEC network performance. The suggested approach makes use of FL to boost learning performance and DRL to learn the best resource allocation strategy for MEC networks.

A meta-reinforcement learning (meta-RL) strategy for quick adaptive task offloading in edge computing networks is suggested in the paper [77]. The essay starts out by outlining the difficulties edge computing networks confront, such as resource limitations and the demand for effective task offloading. The authors explain how quick adaptive tasks offloading with meta-RL can help address these issues. The suggested method considers the dynamic nature of the network and the variable requirements of various applications to learn the best task-offloading strategy for edge computing networks. Using a meta-gradient descent training algorithm, the approach's agents may learn the best strategy for work offloading over numerous episodes.

The paper [78] proposes a hybrid approach that combines deep reinforcement learning (DRL) and imitation learning (IL) for cloud resource scheduling. The article begins by introducing the challenges cloud resource scheduling faces, including balancing resource utilization and user satisfaction. The authors then describe how DRL and IL can be used to address these challenges by enabling intelligent resource allocation and improving the performance of cloud resource scheduling. The proposed approach uses DRL to learn the optimal resource allocation strategy for cloud resource scheduling and IL to improve the performance and stability of the learning process. The agents in the approach are trained using a deep Q-learning algorithm with experience replay, which allows them to learn from past experiences and improve their decision-making over time.

The paper [79] proposes a hybrid approach that combines anomaly detection and deep reinforcement learning (DRL) for resource scaling in cloud computing environments. The article begins by introducing the challenges cloud computing environments face, including the need to balance resource utilization and performance. The authors then describe how anomaly detection and DRL can address these challenges by enabling intelligent resource scaling and enhancing the performance of cloud computing environments. The proposed approach uses an anomaly detection algorithm to identify abnormal resource usage patterns, which are used to trigger resource scaling. The scaling decisions are then made using a DRL algorithm that learns the optimal resource allocation strategy based on the current state of the system and the desired performance objectives.

The paper [80] provides an overview of mist computing's use in Industry 4.0 and smart cities in Africa. The constraints of standard cloud computing models in the situation of smart cities and Industry 4.0 are introduced in the first section of the article, including latency, bandwidth restrictions, and security issues. The authors explain how mist computing, which allows processing and data storage at the network's edge, might be utilized to overcome these issues. The paper provides an overview of the key features and benefits of the mist computing, including low latency, high bandwidth, and improved security. The study then investigates mist computing's possible uses in Africa's smart cities and Industry 4.0., such as intelligent transportation systems, healthcare, and agriculture. The authors provide several case studies and

examples of African mist computing projects, highlighting their impact and potential for future development.

The paper [81] recommends allocating fog computing to edge nodes in mobile contexts utilizing a mobile-embedded platform as a service (mePaaS). The study begins by outlining the difficulties that conventional Cloud computing models encounter in mobile settings, such as latency, bandwidth restrictions, and energy usage. The authors explain how data processing and storage at the network's edge can be made possible through fog computing, which can be utilized to address these issues. The proposed mePaaS platform is designed to distribute fog computing to edge nodes in mobile environments, such as smartphones and wearable devices. The middleware layer, application layer, and infrastructure layer make up the three layers of the platform. The infrastructure layer contains the hardware and software components needed to support fog computing on mobile devices, while the middleware layer provides the necessary networking and communication services. The application layer consists of the user-facing applications and services enabled by the mePaaS platform. To evaluate the performance of the mePaaS platform, the authors conducted experiments using a testbed composed of multiple mobile devices. The results show that the platform can effectively distribute fog computing to edge nodes in mobile environments, reducing latency and energy consumption compared to traditional Cloud computing models.

According to the paper, incorporating self-awareness and attention mechanisms in fog and mist computing systems may have certain advantages [82]. The concepts of self-awareness and attention, as well as how computing systems can use them, are introduced in the study. The authors then describe the challenges traditional fog and mist computing systems face, including limited processing power, energy consumption, and limited communication bandwidth. The paper proposes using self-awareness and attention mechanisms to address fog and mist computing systems challenges. Self-awareness mechanisms allow the system to monitor and analyze its behavior and performance. In contrast, attention mechanisms enable the system to allocate resources dynamically and prioritize tasks based on their importance. The authors provide several examples of how self-awareness and attention mechanisms can be applied in fog and mist computing systems, such as optimizing resource allocation in real time and adapting to changing environmental conditions. The paper also discusses the potential benefits of incorporating these mechanisms, including improved performance, energy efficiency, and reliability.

In addition to presenting a case study of the THINNECT platform as an example of a mist computing system, the paper [83] outlines how mist computing developed from fog and cloud computing. The concept of fog and cloud computing is introduced in the first section of the study, along with the drawbacks of these established paradigms in terms of latency, bandwidth, and scalability. The authors then review how mist computing, a combination of cloud and fog computing, can help overcome these constraints by allowing data processing and storage at the network's edge. The paper then presents a case study of the THINNECT platform, which is a mist computing system designed for the industrial IoT (IIoT) market. The THINNECT platform consists of multiple layers, including a hardware layer, a communication layer, a middleware layer, and an application layer. The hardware layer contains sensors and gateways that collect and transmit data, while the middleware layer provides the necessary networking and communication services. The application layer consists of the user-facing applications and services enabled by the THINNECT platform. The authors describe the key features and capabilities of the THINNECT platform, such as its support for low-latency communication, real-time data processing, and secure data storage. The paper also presents several case studies demonstrating the platform's effectiveness in industrial settings, such as monitoring and controlling energy consumption in factories and optimizing production processes.

The study [18] outlines prospects for research in this field and offers a thorough taxonomy of scheduling algorithms for IoT applications in edge and fog computing environments. The authors then provide a taxonomy of scheduling strategies divided into groups based on their goal (e.g., maximizing throughput, minimizing energy use) and the kind of IoT application. (e.g., data collection, data processing, control). The paper describes each scheduling technique, including its strengths and weaknesses, and highlights the key research challenges and unresolved concerns in this field of study. The authors also discuss the potential benefits of using machine learning and artificial intelligence techniques to improve scheduling in edge and fog computing environments. Finally, the paper identifies several future directions for research in this area, such as developing more efficient and accurate scheduling algorithms, addressing the challenges posed by heterogeneity and scalability, and exploring the potential of combining multiple scheduling techniques.

On the Internet of Things (IoT) context, the paper [19] begins by outlining the idea of IoT and how it has the potential to produce enormous volumes of data, necessitating solutions for computation and storage that are effective and scalable. They also discuss the challenges posed by integrating these paradigms with IoT, such as the need for low-latency communication and the need to address the heterogeneity of IoT devices. The identified emerging possibilities include Real-time data analytics: fog and mist computing can enable real-time processing of data generated by IoT devices, allowing for faster decision-making and improved efficiency, Resource optimization: cloud, fog, and mist computing can optimize computing and storage resources, improving scalability and reducing costs, and Security: fog and mist computing can improve safety by allowing for distributed and decentralized data processing and storage.

**TABLE 4.** A survey of surveys (SoS) on various computing paradigms.

| Reference | Time Interval | Systematic Framework | Work Done | ML/DL Model-based Task Offloading | Computing Paradigms | | | Future Scope | Articles Considered |
|---|---|---|---|---|---|---|---|---|---|
| | | | | | EC | FC | CC | | |
| [2] | 2010-2022 | N | A state-of-the-art load balancing review is presented for the mist, fog and cloud computing, highlighting the various RQs, challenges and future directions. | N | N | Y | Y | Y | 204 |
| [18] | 2012-2021 | Y | A taxonomy of current research on scheduling IoT programs in fog computing is presented in this paper, highlighting current research gaps and highlighting potential future directions. | N | N | Y | N | Y | 144 |
| [19] | 2014-2021 | N | This paper describes the layer-wise construction of various computing skills as well as the rising prospects of mist, fog, and cloud computing in IoT models. Additionally, the IoT paradigm's numerous qualities and problem-solving importance have been thoroughly examined. | N | Y | Y | Y | N | 58 |
| [20] | 2012-2020 | N | This paper offers a thorough study of the ways in which the cloud and/or edge can be joint to solve the task offloading issue. The many aims, restrictions, and dynamic circumstances of this end-to-end application accomplishment strategy are partly satisfied by using control theory optimization approaches, mathematical, and artificial intelligence. The review's conclusion finds unanswered issues and possible answers for the matter at hand. | Y | Y | N | Y | Y | 294 |
| [5] | 2009-2021 | N | This paper analyzes the changing computing paradigms thoroughly and illustrates the unresolved research problems while emphasizing future directions. | N | N | Y | Y | Y | 181 |
| [9] | 1998-2021 | Y | The most recent analysis of ML/DL-based resource management techniques for cloud computing is presented, outlining any unresolved issues as well as the benefits and drawbacks of each method. Additionally, it emphasizes potential avenues for further study in light of outstanding problems. | Y | N | N | Y | Y | 117 |
| [21] | 2014-2020 | Y | It highlights the research deficit in this area by providing an analytical overview of the current metaheuristic and heuristic load-balancing strategies in cloud computing. Additionally, each algorithm's flowchart is presented in this review. | N | N | N | Y | Y | 30 |
| [1] | 2009-2018 | Y | This paper examines the most recent hyper-heuristic, hybrid, meta-heuristic, and heuristic load balancing methods used in cloud computing. | N | N | N | Y | Y | 125 |
| [22] | 2009-2020 | N | Examining the ideas, tools, architecture, and procedures in the IoT-Cloud-Fog continuum gives a taxonomy of the emergent challenges. | N | Y | Y | Y | Y | 102 |
| [23] | 2010-2020 | N | The effectiveness of the cloud, fog, and edge paradigms are evaluated in this review using a taxonomy of the QoS parameters. | N | Y | Y | Y | Y | 50 |
| [24] | 1999-2020 | N | The design and management of a fog computing architecture are highlighted in great depth. It also provides open research problems and prospective trends. | N | N | Y | N | Y | 194 |
| [25] | 2002-2019 | N | A cutting-Edge computing paradigm is given with a focus on fog computing, its planning, and a arrangement of load-balancing methods. | N | Y | Y | Y | Y | 191 |
| [26] | 2005-2020 | N | This paper summarizes the study on edge computing's key structures, By outlining its key characteristics, addressing privacy and security alarms in the cloud, and showcasing edge computing technologies | N | Y | N | Y | Y | 62 |
| [27] | 2009-2019 | N | This paper discusses cloud and edge computing principles, architecture, techniques, research gaps, and bottlenecks. | N | Y | Y | Y | Y | 101 |
| [28] | 2004 | N | This paper focuses on the essential characteristics and structures | N | Y | Y | Y | Y | 71 |

**TABLE 4.** *(Continued.)* **A survey of surveys (SoS) on various computing paradigms.**

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | - 2019 | | of the cloud, fog, and edge computing paradigms. Furthermore, it demonstrates the importance and interdependence of the IoT-Cloud-Fog-Edge continuum. | | | | | | |
| [29] | 2011 - 2019 | N | Fog computing is presented in terms of its work and significance with edge computing by highlighting ML/DL-dependent resource management strategies and safety considerations. | Y | Y | Y | N | Y | 95 |
| [30] | 2001 - 2018 | N | A organized and thorough analysis highlights the fog and its related computer paradigms. It also emphasizes the difficulties in the research and ends with a discussion of future directions. | N | Y | Y | Y | Y | 450 |
| [31] | 2012 - 2018 | N | The technology and difficulties of edge computing are discussed in this paper, along with the worries that come with switching from cloud to edge computing. | N | Y | Y | N | Y | 114 |
| [32] | 2012 - 2017 | N | A modern fog computing analysis encompasses features, architecture, and benefits. It finishes by discussing how fog computing complements cloud and IoT technology. | N | N | Y | N | Y | 63 |
| [33] | 1991 - 2018 | N | It provides a thorough analysis of the techniques developed as well as several new computing paradigms, including IoT, fog, cloud, and mobile computing. | N | Y | Y | Y | Y | 412 |
| [34] | 1997 - 2017 | N | This paper examines resource distribution and management techniques for fog computing in order to minimize causes of delays, bandwidth, and power usage. | N | N | Y | N | Y | 225 |
| [35] | 2006 - 2017 | Y | A thorough review of fog computing is presented, importance components, construction, methods, and linked challenges, as well as open research challenges and forthcoming directions. | N | N | Y | N | Y | 168 |
| [36] | 2001 - 2018 | N | The presentation of a thorough study of fog computing highlights the characteristics, architecture, tactics, and related difficulties with open research problems and future goals. | N | Y | Y | Y | Y | 142 |
| [37] | 2003 - 2017 | N | A comprehensive review of multi-access-edge computing for mobile ad-hoc cloud computing, including announcement and resource management, is presented. | N | Y | N | Y | Y | 242 |
| [38] | 2001 - 2017 | N | A thorough overview of technology based on IoT, its designs, associated concepts involving confidentiality issues, and the fusion of the fog and IoT computing standards are presented in this paper. | N | Y | Y | N | Y | 167 |
| [39] | 2012 - 2016 | N | The literature review gaps are discussed together with a description of the fog architecture, related services, and problems. | N | Y | Y | Y | Y | 47 |
| [40] | 2001 - 2017 | N | A relative analysis of fog computing with further rapidly growing paradigms for computing is given by describing the essential skills and arrangement concerns. | N | Y | Y | Y | Y | 123 |
| [41] | 1999 - 2016 | N | It investigates the evolutionary-dependent IoT and also its involvement in the paradigms for computing in order to ensure the challenges associated with it. | N | Y | Y | Y | Y | 119 |
| [100] | 2011- 2022 | Y | A state-of-the-art task offloading review has been presented for six different computing paradigms, such as IoT, dew, mist, edge, fog, cloud, and osmotic. It highlights the RQs, future directions and open issues. | N | Y | Y | Y | Y | 181 |
| [104] | 2013 - 2022 | Y | This review offers a comprehensive understanding of the applications and limitations of each of these three computing architectures, along with how IoT integration fits into them. To progress the Internet of Things, it also outlines the benefits, open challenges, potential future applications, and future directions of IoT integration with computing systems. | N | Y | Y | Y | Y | 194 |
| *EC: Edge computing, FC: Fog Computing, CC: Cloud Computing* | | | | | | | | | |

Authors in [84] proposed an I-apriori algorithm as a Classification mining algorithm to achieve more efficient scheduling of tasks in a fog environment. In [85], both task and network conditions are taken to reduce the response time and improve effectiveness. To reduce the geographical delay, a distance-based delay approach is devised for an even distribution of loads. Furthermore, a load balancing algorithm is incorporated to have an equal distribution of load. Gazori et al. [86] focused on task orchestration for IoT-Fog services with the goal of lowering long-term service

**TABLE 5.** A comparison of existing SURVEY OF articles on the review criteria.

| Reference & Year | Relative Study | The range of Algorithms Considered | Flow Diagram | Graphs | Classification | Proposed Architecture | Scheduling Metrics | Articles Considered |
|---|---|---|---|---|---|---|---|---|
| [2], 2023 | Yes | Yes | No | Yes | No | No | Yes | 103 |
| [100], 2023 | Yes | Yes | Yes | Yes | Yes | Yes | Yes | 181 |
| [104], 2023 | Yes | Yes | Yes | Yes | Yes | Yes | Yes | 194 |
| [18], 2022 | Yes | Yes | No | No | Yes | No | Yes | 144 |
| [21], 2022 | Yes | Yes | Yes | Yes | Yes | No | Yes | 136 |
| [19], 2021 | Yes | No | No | No | No | Yes | Yes | 58 |
| [20], 2021 | Yes | No | No | Yes | No | Yes | Yes | 294 |
| [1] 2020 | Yes | Yes | No | Yes | Yes | No | Yes | 125 |
| [42], 2020 | Yes | Yes | Yes | Yes | No | No | No | 21 |
| [43], 2020 | Yes | Yes | No | Yes | Yes | No | Yes | 55 |
| [44], 2019 | Yes | Yes | No | Yes | Yes | No | Yes | 127 |
| [45], 2019 | Yes | No | Yes | No | Yes | No | No | 35 |
| [46], 2019 | Yes | Yes | No | Yes | Yes | No | Yes | 71 |
| [47], 2019 | Yes | Yes | No | No | No | No | No | 38 |
| [48], 2018 | No | Yes | No | No | No | No | No | 11 |
| [49], 2018 | No | Yes | No | No | No | No | No | 14 |
| [50], 2018 | Yes | Yes | No | No | No | No | No | 19 |
| [51], 2018 | Yes | No | No | No | Yes | Yes | No | 33 |
| [52], 2017 | Yes | Yes | No | No | Yes | No | No | 14 |
| [53], 2017 | No | No | No | Yes | No | No | No | 15 |
| [21], 2022 | Yes | No | Yes | No | Yes | No | No | 136 |
| [54], 2017 | No | No | No | Yes | No | No | Yes | 15 |
| [55], 2017 | No | Yes | No | No | No | No | Yes | 47 |
| [56], 2017 | Yes | Yes | Yes | Yes | Yes | No | Yes | 162 |
| [57], 2017 | Yes | Yes | No | No | No | Yes | Yes | 31 |
| [58], 2017 | No | No | No | Yes | No | No | Yes | 15 |
| [59], 2017 | No | Yes | No | No | Yes | No | No | 27 |
| [60], 2017 | Yes | Yes | No | Yes | Yes | No | Yes | 108 |
| [61], 2016 | Yes | Yes | Yes | Yes | Yes | No | Yes | 84 |
| [62], 2016 | No | Yes | No | Yes | No | No | No | 12 |
| [63], 2016 | No | Yes | No | No | No | No | No | 19 |
| [64], 2015 | Yes | Yes | No | Yes | No | No | Yes | 25 |
| [65], 2015 | Yes | Yes | No | No | No | No | No | 11 |
| [66], 2015 | Yes | Yes | No | Yes | Yes | No | Yes | 64 |
| [67], 2015 | Yes | Yes | No | No | No | No | No | 96 |
| [68], 2014 | Yes | Yes | No | No | No | No | Yes | 10 |
| [69], 2014 | No | No | No | Yes | No | No | No | 16 |
| **Current Survey** | Yes | Yes | Yes | Yes | Yes | Yes | Yes | 109 |

latency and compute cost while working within resource and schedule constraints. To address this problem, the authors have provided a scheduling strategy based on Double Deep Q-Learning (DDQL), a target network, and experience replay techniques. To orchestrate the workloads that are privacy-sensitive effectively while balancing the workloads, Razaq et al. [87] classified IoT tasks in terms of their privacy requirements, real-time needs, and processing time. They also divided IoT requests into minute chunks based on their security requirements. To solve this, the authors developed an RL approach where the agent attempts to adhere to the required privacy and delay criteria while avoiding

**TABLE 6.** A comparison of ML/DL-focused load balancing algorithms with future directions.

| Ref. | Research Objectives | Algorithm & Type | Computing Paradigm | Strengths | Scope |
|---|---|---|---|---|---|
| [70] | Lowering costs | MDP_DT, Adaptive Reinforcement Learning (ARL) | Cloud | Splits the state space adaptively using statistical techniques to provide precise splits. | To combine MDP_DT and DQN methods in order to tackle complicated challenges |
| [72] | Flexible provisioning | DERP, DL w/o RL | Cloud | Although DERP has three components, space partitioning is not necessary for it to collect rewards. | To develop an intercommunicate architecture comprising simple DRL, full DRL, and DDRL; to combine federate learning with DERP. |
| [73] | Reduction of energy costs and consumption | DRL_Cloud, DRL | Cloud | Exploration and exploitation, target networks, and experience replay can speed up convergence | To apply it to both static and dynamic scheduling |
| [74] | Minimising response time and increasing CPU usage | QEEC, Q-Learning (RL) | Cloud | The suggested algorithm decreased task wait times on average, and the dynamic task ordering strategy improves the quality of Cloud services. | To construct alternative queueing models to satisfy realistic instances and to use Meta-Heuristics to increase performance |
| [75] | Reduce energy use while enhancing QoS | AGH+QL, RL | Cloud | Training was accelerated via anchor graph hashing (AGH), and hash codes made the state space smaller. | To combine AGH and DRL |
| [76] | Decrease in makespan and load imbalance | DQST, DQL | Cloud | A high-quality bi-objective optimisation solution can be obtained using the entropy weight method. | To develop a framework with many objectives or a model of energy use; to consider dependent tasks |
| [77] | Reduction of energy usage and reaction time | MDRL, DRL | Cloud | Reducing the underlying practical issues when DRL is applied in scalable state space is ensured by uniform resource mapping. | To take workflow and related tasks into account |
| [78] | Reduction of the makespan | RLTS, DRL | Cloud | The relationship between state-agent and action is stated using the DQN. | NA |
| [79] | Reduction of service delay and energy consumption | IDRQN, DRL | Mobile-Edge | Candidate networks to decouple action selection and action value assessment; LSTM to estimate value | Transfer learning might be used to create heterogeneous MEC, while federate learning could be used to address challenges with multiple objectives. |
| [80] | Reducing calculation time, and increasing channel utilisation | MADRL, DRL | Mobile-Edge | Actor-critic and DQN together can speed up the performance | The network's gated recurrent units could be utilised to forecast channel conditions. |
| [81] | Lowering of service latency | DDQN, DRL | IoT-Edge | DDQN can maintain consistency in service delay | could be used to reduce problems like energy efficiency. |
| [82] | Decrease in makespan and load imbalance | DRL+FL, DRL | Mobile-Edge | Performance can be accelerated in training when DRL and FL are combined. | Other scenarios could make use of the FL/DRL hybridization. |
| [83] | Network traffic and service delay reduction | MRLCO, DRL | Edge | The offloading policy is represented by a neural network using a new training method incorporating first-order approximation. | Utilizing an adaptive client selection technique to sift out stragglers automatically |
| [18] | Speeding up turnaround | DeepRM_Plus, DRL | Cloud | Using learning imitation to accelerate convergence and CNN to gather resource state | To use additional techniques, such as DDPG and the Actor-Critic Network, to rationally identify the state |

**TABLE 6.** *(Continued.)* **A comparison of ML/DL-focused load balancing algorithms with future directions.**

| | | | | | |
|---|---|---|---|---|---|
| [19] | Response time reduction and increased CPU efficiency | ADRL, DRL | Cloud | Utilizing an anomaly detection model to identify performance issues and increase environmental awareness | To speed up training, it is possible to use a parameter-tuning technique and a mix of DRL and semi-supervised learning. |
| [84] | Execution time, AWT | I-Apriori | Fog | Devised an I-apriori algorithm as a Classification mining algorithm to achieve more efficient scheduling of tasks in a Fog environment. | The suggested algorithm has only been tested in a limited simulation. Security and privacy concerns are not addressed. |
| [85] | Response time, Scheduling time, Load-balancing rate, Delay, Energy consumption | Delay-aware load balancing technique | Fog | Both network conditions and tasks are taken to reduce the response time and improve effectiveness. A distance-based delay approach is devised to reduce geographical delays. Furthermore, a load balancing algorithm is incorporated to have an equal distribution of load. | The suggested architecture assumes the availability of the Fog nodes, which may not be the case in practical situations. The effectiveness of the proposed framework in comparison to current methods in terms of latency, throughput, and other metrics has not been assessed by researchers. |
| [86] | Avg. Computation cost, Deadline, Avg. Energy consumption, Finished time, Makespan, Response time, Dealy, Waiting time | DDQL | Fog | Devised a task scheduling method using DDQL and formulated the assignment problem through the Markov decision model. | The recommended methodology might not be feasible in certain cases where resources are scarce because it requires a significant amount of training resources to train the DRL model. |
| [87] | Delay, Security violation, Load balancing rate | Q-learning | Fog | proposes a Q-learning-based strategy that instructs the computer to schedule impending work on servers located in the Cloud or Fog layer. | The suggested method assumes that the Fog nodes are fully and accurately informed on their own state as well as the tasks they are currently working on. The performance of the suggested strategy may have been more accurately assessed by the authors had they compared it to any state-of-the-art approaches. |
| [88] | AWT, Service latency, Deadline | BLWJaya | Fog-Cloud | Proposes a latency-aware task offloading strategy with task prioritization using Fuzzy logic. | Could be validated for other QoS metrics. |
| [89] | Service rate, Utilization, Latency rate, Energy consumption | FedDQL | Dew-Fog-Cloud | Proposes a federated learning-supported task-offloading technique for secured offloading in a dew-enabled paradigm. | Could be validated for other QoS metrics. |
| [101] | Service rate, Utilization, Latency rate, Energy consumption, load balancing rate | Fuzzy logic and BLWJaya | IoT-Fog-Cloud | Utilize a Fuzzy logic algorithm to determine target layers for offloading based on resource heterogeneity and system requirements (network bandwidth, task size, resource utilization, and latency sensitivity).<br>Propose a Binary Linear-Weight JAYA (BLWJAYA) task scheduling algorithm to map incoming IoT requests to computation-rich Fog nodes/VMs. | Could be validated for other QoS metrics. Task dependency could also be considered. |

overloading specific fog nodes to decrease overall latency. Authors in [88] devised an offloading algorithm using a binary variant of linear weight (LW) Jaya considering task prioritization using a Fuzzy method. In addition, this work considers latency overhead as a key scheduling metric and gains a significant reduction along with other scheduling metrics. To devise a computationally and energy-efficient task offloading algorithm, authors in [89] utilized a federated learning-supported deep Q-learning approach in a Dew-enabled vehicular Fog computing. To safeguard the data and preserve data privacy, a federated learning concept is integrated into architecture. Dew-enabled devices are proposed to make a safe and secure connection while offloading tasks from one vehicle to another for processing in unstable network connectivity. Finally, the research [101] focuses on optimizing task offloading and load balancing in Fog-Cloud systems for IoT applications, leveraging fuzzy logic and a specific scheduling algorithm to improve various performance metrics (resource utilization, service rate, latency rate, energy consumption, and load balancing rate). It utilizes a Fuzzy logic algorithm to determine target layers for offloading based on resource heterogeneity and system requirements (network bandwidth, task size, resource utilization, and latency sensitivity). It proposes a Binary Linear-Weight JAYA (BLWJAYA) task scheduling algorithm to map incoming IoT requests to computation-rich fog nodes/VMs. The experimental results suggest notable enhancements compared to other baseline methods.

## VI. OPEN RESEARCH ISSUES IN FOG COMPUTING

Fog computing exhibits a numeral of technical open issues, including the need for a feasible solution for resource management and scheduling, security and privacy concerns, and challenges with mobility, scalability, reliability, heterogeneity, and sustainability. Here, the author in [6] attempts to summarise the open research issues by categorising them into twelve sectors. These research issues pertaining to fog computing are shown in Figure 7 [6].

1. **Fog Frameworks:** it is referred to as a reference architecture to set up a fog environment for processing users' requests and providing on-demand services for resource-constrained objects [90]. To do so, Cisco proposed an application environment to execute these services on hardware like routers or switches, enabling the computation closer to the edge and leveraging certain limitations of cloud like latency. Over the years, many Fog frameworks have been proposed, such as FogBus [91], Indie Fog [92], iFogSim [93], etc.
2. **Simulators:** these enable the users to deploy a fog set-up on a large scale to simulate disparate workloads with several requirements and real-world configurations. One of the most widely used simulators is iFogSim [93], which is developed and used extensively to execute resource management policies and placement methods in a fog environment. It is configured on top of the CloudSim, a simulator for providing cloud computing services, and inherits the core features of the underlying simulator. Subsequently, it is expanded to iFogSim2 to facilitate mobility and Edge-to-Fog integration. In the context of mobility, another simulator called MobFogSim [94] was developed, which supports live migration from one fog node to another or to a cloud VM. EdgeCloudSim [95] runs on top of CloudSim to provide resource management and orchestration for edge computing. To support network characteristics, FogNetSim++ [96] has been developed. However, these simulators are solely for performing one of the core tasks, and hence, the essence would be lost on integration into one for the sake of ease and convenience and might bring more complexities.
3. **Resource Management and Task offloading:** Services of fog computing fall into two categories: (a) Top-down service, and (b) Bottom-up service. When using top-down services, the cloud provider delivers the required geo-distributed fog resources to control how the services are running throughout the fog topology. Diverse QoS factors, such as user load, proximity, cost models, latency, etc., can be used to provide and manage resources. Services are maintained by distinct fog service providers (FSPs) in the bottom-up method. Different suppliers here offer the resources for the Fog. The gateway devices identify these nearby resources and schedule fog-related actions. Therefore, the responsibility for resource identification, provisioning, scheduling, and placement lies with the supplier of the fog application or service. Fog computing typically involves network, processing, and storage resources. It is necessary to first identify the fog nodes that are supplying the required resources. To decrease geographic and network distances, the location of the fog nodes should be known. FSPs should also be aware of the context, considering the needs of the fog application, the load on nearby fog nodes, and the available resources. There is much literature addressing this issue with intelligent strategies. However, the techniques should primarily be multi-objective while also considering other QoS elements, including scalability, privacy and security, real-time interactions, and device compatibility.
4. **Optimal placement of requests:** the primary concern of this issue is to identify a compatible fog node to place the request. Scheduling and offloading the fog requests are extensively studied in literature and their primary focus is to meliorate the QoS scheduling parameters such as throughput, latency, resource utilization, energy consumption, deadline, etc. In this scenario, the requests or tasks can be categorized into two types: (a) Independent, where each task is considered as a single entity and processed individually on a computing node, and (b) Dependent, where a task is sub-divided into many sub-tasks and processed separately on many computing nodes and later assembled to form a single entity. However, in the latter case, the interdependency among tasks is considered, and each sub-task is processed sequentially if the processing is

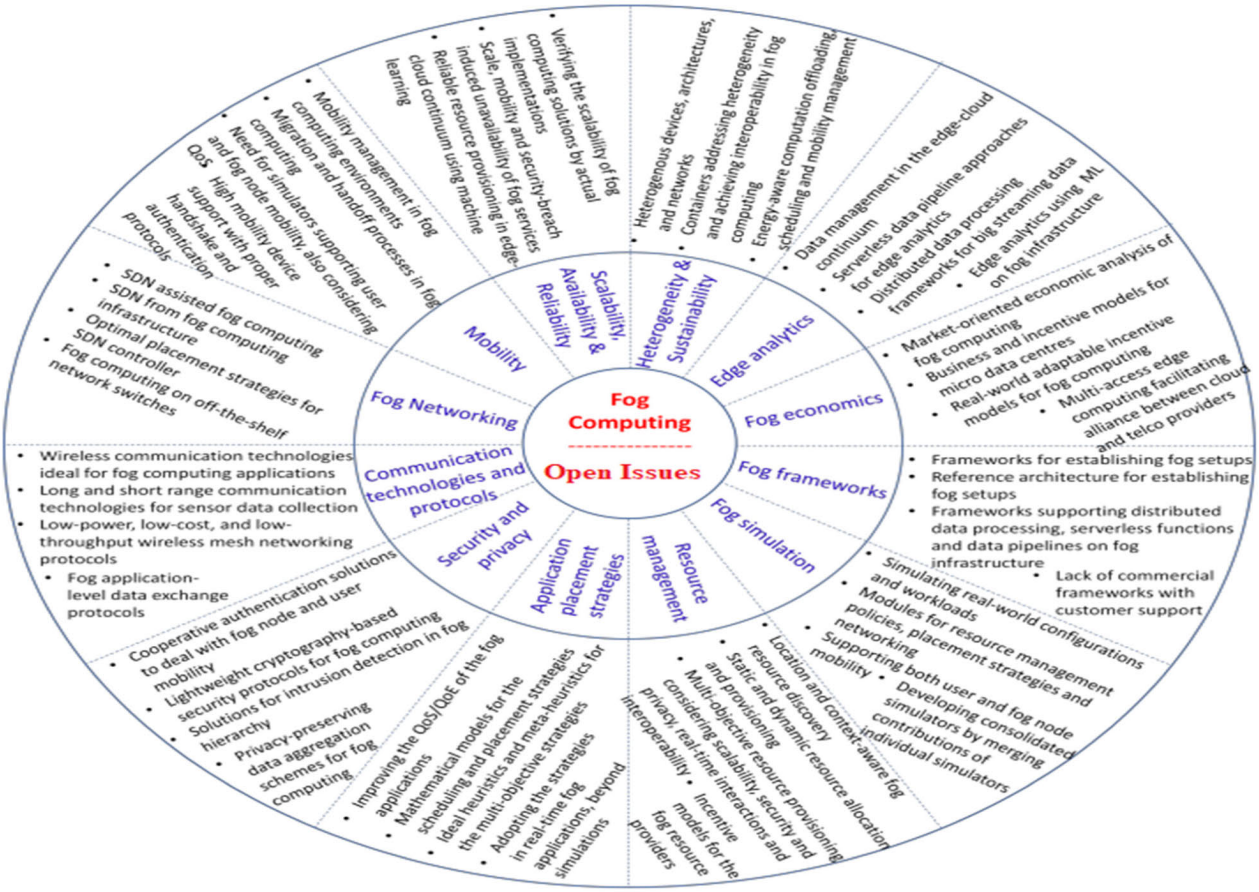



**FIGURE 7.** Open research issues in fog computing [6].

done on a single node. Moreover, the computing resources are also categorized into types, such as Homogeneous, where every resource is of the same configurations and Heterogeneous, where every resource is of different configurations. Different strategies such as machine learning, deep learning, hyper-heuristic, meta-heuristic, and heuristic were investigated to address this issue to keep the core objective intact from both users' perspectives (deadline, response time, priority) and service provider's perspective (high resource utilization, high profit, low power consumption, low CO2 emission). However, most of the Fog deployment options were presented in simulators, and only a limited number have been proven in small-scale real-world applications. There is still much to be gained in the area of placing approaches implementation in real-time fog applications.

5. **Privacy and Security:** Since fog computing typically uses low-resource devices at the edge of the network to supply processing, storage, and network services, it has many security and privacy risks. Fog computing, in the opinion of most, is more secure than cloud computing. Fog gathers and analyzes sensor data locally, reducing reliance on the cloud. By doing this, dangers from the internet like man-in-the-middle assaults are removed and eavesdropping. Since the user data remains within the premises, the confidentiality of the data is also improved. However, threats like manipulation, jamming, and denial-of-service are more likely to target communication between edge and fog devices. Because of the fog devices' resource limitations, the issue is further mitigated because of the implementation of a few common security mechanisms, such as encryption, blockchain and cryptography. Fog computing poses another security breach in the form of malevolent attackers trying to hack the fog system.
6. **Communication enablers:** The devices in the hierarchical fog computing architecture can communicate and connect in four distinct ways thanks to edge nodes, fog nodes, and cloud datacenters with the hierarchical IoT-Fog-Cloud continuum (Figure 2). The four different types of connectivity are (a) A wireless connection between the IoT devices and the Fog nodes, and the IoT devices and the cloud datacentre, (b) A wired/wireless connection between the fog nodes and the cloud data centre, (c) A wireless connection amongst the edge devices and the fog nodes, and (d) A wired/wireless connection between the fog nodes and the physical hosts in cloud. These

connections and communications are really made possible by a number of communication technologies and protocols that have developed throughout time. In order to support IoT/fog applications, the technologies are primarily based on optimising energy efficiency, communication range, standards compliance, bandwidth, etc.

7. **Networking:** The zettabytes of sensor data generated from the billions of internet-enabled devices are transferred to the cloud for processing, clogging the main network and increasing latency. By processing the data in proximity, fog computing solves this problem. Fog computing does not, however, completely take the role of clouds. Because of their complementary nature, cloud and fog computing must be used in conjunction with another ground layer in a variety of applications. As a result, to coordinate the connection between fog and cloud servers, a middle networking layer is needed. The orchestration of this intermediary networking layer can be done using the developing computer networking paradigm known as software-defined networking (SDN).
8. **Mobility:** Given the high degree of mobility of many fog devices, such as smartphones, vehicles, and drones, one of the most interesting challenges in fog computing is mobility. The edge devices that supply the sensor data and act as fog consumers in fog applications, as well as the fog devices that carry out the intermediate processing, can both be mobile. Roadside equipment can serve as static fog nodes in applications like smart cars, where the vehicles are mobility. In the case mentioned above, fog computing performance is hampered by edge or fog node movement. The QoS (latency, scalability, reliability, etc.) and security of fog applications are greatly reduced by mobility. For this reason, fog computing has trouble supporting high-mobility devices.
9. **Scalability, Availability and Reliability:** it is evident from much literature that fog nodes are not scalable to the growing demands of users leading to the unavailability of fog nodes for processing billions of tasks. The mobility issue of fog computing also paves the way for the unavailability of resources. Both issues open the door to unreliability. The reliability factor deals with the failures and recoveries of ongoing tasks on certain nodes. Due to the non-scalable and unavailability of sufficient resources, the failed tasks cannot be offloaded to other fog nodes.
10. **Task and Resource Heterogeneity:** tasks generated by disparate IoT devices are of different lengths and conflicting requirements and need to be processed by heterogeneous resources to prevent starvation and aging.
11. **Edge analytics:** data management and processing across the edge and fog nodes are referred to as edge analytics. It is basically conducted through ML/DL techniques.
12. **Fog Economy:** Bringing cloud services (networking, computation, and storage) close to IoT and edge devices is the aim of fog computing. Network transmission costs and latency will be minimized by processing the data at the point of creation. Latency reduction indirectly increases labor efficiency by supporting more users and apps, which enhances user experience and leads to revenue growth. Furthermore, the possibility of a complete service interruption is decreased because the distributed fog lessens reliance totally on centralised clouds.

### A. INTERDEPENDENCIES AMONG RESEARCH ISSUES

In addition to identifying individual research issues, it is essential to understand the interdependencies among them, as fog computing challenges are not isolated but deeply interconnected. For instance, resource heterogeneity (Issue 10), which refers to the diverse computational capabilities, architectures, and configurations across edge and fog nodes, directly exacerbates security vulnerabilities (Issue 5). The lack of uniformity in hardware and software stacks introduces inconsistencies in security policies, leaving some devices more susceptible to attacks and reducing the overall robustness of the fog environment.

Moreover, mobility (Issue 7) significantly impacts task offloading and service placement (Issue 1) as well as QoS assurance (Issue 3). When users or nodes move dynamically, previously optimal placements may become suboptimal or invalid, causing service interruptions and increased latency, thus violating QoS expectations. Similarly, energy constraints (Issue 4) are closely tied to real-time analytics (Issue 2) and resource scheduling (Issue 6), where computationally intensive tasks must be balanced with battery and energy limitations, especially in mobile and IoT devices.

Understanding these interrelationships helps identify cross-cutting research themes, such as context-aware orchestration, holistic security frameworks, and dynamic service migration strategies, which can address multiple challenges simultaneously. Recognizing such dependencies enables researchers and practitioners to better prioritize efforts in designing scalable, resilient, and intelligent fog systems.

## VII. RESEARCH QUESTIONS AND INFERENCES

**RQ 1.** *How can reinforcement learning be leveraged to optimize task scheduling and resource allocation in the IoT-Fog-Cloud continuum?*

Reinforcement learning (RL), especially Deep Q-Networks (DQN) and Actor-Critic methods, can predict optimal offloading policies by learning from system dynamics like task arrival rates, resource availability, and latency constraints. Techniques like Federated Reinforcement Learning (FRL) can enable decentralized decision-making, further enhance scalability and reduce communication overhead between layers.

**RQ 2.** *What role do hybrid DL approaches (e.g., combining CNNs and LSTMs) play in predicting and reducing network congestion in fog environments?*

Hybrid approaches, such as combining CNNs for feature extraction and LSTMs for sequence prediction, can model temporal variations in task loads and network congestion patterns. These methods can optimize task distribution by proactively reallocating workloads to less congested

fog nodes, thereby minimizing latency and ensuring QoS adherence.

**RQ 3.** *Can meta-reinforcement learning address dynamic changes in IoT workloads better than traditional RL in fog environments?*

Meta-reinforcement learning can enhance adaptability by quickly tuning task-offloading strategies based on changing workloads. Unlike traditional RL, meta-RL learns how to learn, enabling faster adaptation to new environments or unexpected surges in IoT device-generated data, leading to more resilient Fog systems.

**RQ 4.** *How do DL-based anomaly detection models contribute to enhancing resource utilization and failure prediction in the Fog-Cloud continuum?*

Deep learning models such as autoencoders or GANs can detect anomalies in resource usage patterns or performance metrics like CPU utilization, memory, and bandwidth. Early detection of anomalies allows for proactive redistribution of tasks, minimizing service delays and preventing node failures.

**RQ 5.** *What are the impacts of task dependencies and deadlines on the efficiency of DL-based task-offloading algorithms?*

DL-based algorithms, incorporating graph neural networks (GNNs), can model task dependencies to prioritize and schedule tasks efficiently under stringent deadline constraints. By understanding task interdependencies, the system can minimize waiting times and avoid bottlenecks in resource-limited environments.

**RQ 6.** *How can the time complexity of ML/DL algorithms be reduced for resource management in Fog-Cloud data centers?*

Time complexity can be reduced by adopting lightweight ML/DL models, such as pruned neural networks or quantized models, which require fewer computations. Alternatively, hybrid algorithms combining heuristic methods with ML/DL models can address resource allocation problems more efficiently. Techniques like reinforcement learning with anchor graph hashing (AGH) or feature hashing can significantly reduce the state space, accelerating decision-making processes in resource-constrained Fog environments.

**RQ 7.** *How can the accuracy of workload prediction be enhanced in ML/DL-based Fog-Cloud task scheduling?*

Accuracy can be improved by integrating hybrid ML/DL models, such as combining CNNs for feature extraction with LSTMs for capturing temporal dependencies in workload patterns. Federated learning (FL) can also be employed to incorporate real-time local data from Fog nodes, enhancing predictions without compromising privacy. Additionally, context-aware approaches that consider network conditions, task priority, and historical data further refine workload predictions.

**RQ 8.** *How can training time be minimized when developing ML/DL models for task offloading in the IoT-Fog-Cloud paradigm?*

Training time can be reduced using transfer learning, where pre-trained models from similar environments are fine-tuned for specific tasks. Techniques like meta-learning and distributed training across fog nodes can further expedite the process. Federated reinforcement learning (FRL) allows decentralized training, reducing centralized computation bottlenecks while leveraging local task data for rapid convergence.

**RQ 9.** *How can energy consumption in Fog-Cloud paradigms be significantly reduced?*

Energy consumption can be minimized by employing deep reinforcement learning (DRL)-based offloading strategies that prioritize energy-efficient nodes. Predictive models like LSTM, Gated Recurrent Units (GRUs) or TinyML can forecast resource usage, enabling proactive scaling and efficient task allocation. Hierarchical fog layers and resource clustering, combined with advanced load-balancing algorithms, can also optimize resource utilization, reducing unnecessary energy expenditure.

## VIII. CHALLENGES AND FUTURE DIRECTIONS

Fog and edge computing have enabled major advancements in AI and ML/DL. This segment focuses on the numerous applications that take scheduling methods into account from the perspectives of various computing paradigms. Additionally, it covers the unresolved problems, upcoming difficulties, and upcoming directions for task offloading in fog computing.

### A. CHALLANGES

- **Fog node Resource Utilization:** The storage, processing, and energy requirements of fog devices are limited. Both latency-sensitive and delay-tolerant apps send them fluctuating workloads. As a result, the difficult part is planning for the unpredictability of tasks arriving on these fog nodes to make the most of the available resources.
- **Dynamic resource allocation based on the quality of service:** In datacenters, the percentage of executing the task on active VMs varies the most. As a result, numerous users attempt to manage the application simultaneously. In fixed resource distribution, the quantity of resources remains constant regardless of how much demand there is. As a result, it results in non-linear resource usage in times of low or high need. As an outcome, the quality of service is poor when there isn't a spike in demand because the resources aren't scalable. In conclusion, the allocation of static resources is ineffective in both short and high-demand scenarios. Dynamic resource provisioning appears to offer a possible solution to this problem. It can be difficult to determine how much resource must be provided in order to satisfy final demand and raise service quality criteria.
- **Fog node Optimal Resource Assignment:** IoT devices produce a lot of tasks, which should be optimally circulated among fog nodes to have quicker response times,

mainly for latency-sensitive applications. The mobility of fog nodes and IoT devices is supported by Fog computing; thus, all the resources available at once might not be accessible at any other moment. As a result, allocating resources becomes difficult [71]. The problems that require attention include slow generalization, slow adaptability of the existing techniques, and long latency for real-time applications.

- **Estimate of multi-layered resources required for dynamic provisioning:** In emerging computing standards, dynamic provisioning is a desirable attribute that automatically scales resources up and down to meet demand spikes. The need for resources to manage dynamic, expanding, contradictory, and multi-dimensional requests from different types of users is difficult. It requires the workload to be predicted in the future and the resources (CPU, memory, storage, network, disc, etc.) to be automatically scaled in response. Each resource type forecasting is exceedingly difficult and complex.
- **Fog node Parallel Scheduling:** A task is divided into several functions in parallel processing, and each of these sub-tasks is then carried out simultaneously. Another unresolved problem that requires attention is the division of activities into smaller tasks that can be accomplished using distributed computing.
- **Fog node Privacy:** Numerous fog applications, including smart healthcare, send a lot of personal data to fog nodes. As a result, users place a high value on data privacy. Even though some experts use fog nodes to smear privacy-preserving procedures, there is no workable validation option. Authentication is a complex problem since fog nodes are more susceptible to possible attackers.
- **Fog Node Security:** One of the most significant issues is security because fog nodes are under-resourced and placed in risky regions, making them vulnerable to assault. Consequently, it is not easy to construct a lightweight, quick, and reliable safety algorithm. Only a few researchers now concentrate on fog computing security challenges, and some unresolved problems, including dynamic confirmation, access controls, external threats, and intrusion recognition, still need to be resolved.
- **Context-aware Service Provisioning for Fog node:** Context makes up the runtime variables that can have an impact on applications. Context-aware service provisioning methods now in use are less adaptable, scalable, and incapable of supporting several IoT applications. To get over the restrictions, more methods for context-aware service delivery should be researched.
- **Fog node energy consumption:** Energy-conscious fog computing is still a problem that must be solved because low-power batteries limit the energy that fog devices can utilize. While some academics concentrate on energy optimization, other concerns like proper bandwidth consumption for data transfer, energy waste, and battery sewage problems still require attention.
- **Fog Node Self-Adaptive Scheduling:** Self-adaptive resource scheduling is a barrier in fog computing task scheduling because most scheduling techniques deficiency the learning component. Self-adaptive scheduling has been considered in some research papers; however, all this hard work has only been undertaken at the practical level. As a result, task scheduling procedures are needed in a dynamic environment to efficiently plan tasks resulting from unforeseen events.
- **VM management based on SLA:** Geographically dispersed IoT devices now generate computationally heavy tasks because of new computing paradigms, which must be completed by the deadline without lowering the service rate. The SLA prerequisite, like a deadline or inactivity, is not considered as these numerous tasks are executed over multiple VMs, leaving one underutilized. While estimating resource utilization, this constraint appears difficult. Additionally, when accommodating peak demand, over-provisioning and under-provisioning in datacenters result in issues. Consequently, one of the biggest challenges for cloud service providers is carefully figuring out how many resources need to be deployed and for how long.
- **Flexible network traffic owing to VM placement:** The current focus of the study is on building and allocating the number of virtual machines to each host according to its ability to handle the demand. Because of the uneven distribution of assignments, which considers dynamic parameters such as duration, magnitude, deadline, urgency, bandwidth, etc., the VMs eventually become overloaded or underloaded. These contradictories yet demanding factors cause the ineffective use of resources. Likewise, network bandwidth also represents a serious hazard when performing calculations over the cloud. The dynamic bandwidth needs dynamically expanding assignments impact the cloud web. As a result, the placement of the VMs during migration is affected, which creates additional problems like SLA violations and performance degradation.

### B. FUTURE DIRECTIONS

- Although contextual information has been considered in various works of fog-based research, many crucial features of contextual information remain undiscovered. A potential area for fog-based research is the examination of multiple approaches to using contextual information in resource and service management.
- Fog nodes sometimes must function as a computing module than a networking module in situations where real-time interactions are involved. In other conditions, the networking abilities of fog nodes take precedence over the computational ones. Thus, a self-customizable, interoperable architecture of fog nodes is crucial in order to meet the criteria.

**TABLE 7.** Classification of future research directions with evaluation metrics.

| Sl No. | Future Direction | Primary Focus | Key Evaluation Metrics |
|---|---|---|---|
| 1 | Self-customizable & interoperable fog node architecture | System design & interoperability | Scalability, Interoperability Index, QoS assurance |
| 2 | Ensemble learning for workload prediction | Prediction & scheduling | Latency reduction, Forecast accuracy, Task completion time |
| 3 | Context-aware caching & task offloading | Latency optimization | Cache hit ratio, Service response time, Network overhead |
| 4 | Lightweight security mechanisms for edge/fog environments | Security & privacy | Encryption overhead, Data breach rate, Authentication latency |
| 5 | Resource-aware service migration strategies | Mobility & adaptability | Downtime, Migration cost, Energy savings |
| 6 | DVFS-enabled energy-efficient resource scheduling | Energy conservation | Energy efficiency, SLA compliance, CPU utilization |
| 7 | Blockchain-based trust management in federated fog systems | Trust & data integrity | Transaction latency, Trust score, Security compliance |
| 8 | AI-driven anomaly detection and fault tolerance mechanisms | Reliability & resilience | Detection rate, Recovery time, False positives |

- To process the requests, every computing node at the edge, fog, or cloud expends some energy. Furthermore, because tasks are dynamic, some of them are extensive or have a broad scope. Processing these advanced, latency- and delay-sensitive tasks is the primary concern. Not only does the CPU need to process these activities, but also storage, memory, networks, bandwidth, etc. As a result, a system that can predict how much each of these resources would be used could help calculate energy usage. Therefore, the ensemble learning techniques may accurately forecast the workload and save energy use.
- To reduce power usage in cloud datacenters, fog computing has been taken into consideration. Investigations into energy usage optimization inside the fog network are still ongoing. Additionally, consolidating fog nodes by moving tasks from an overloaded node to an underloaded node can be helpful in some cases for managing power in a fog system. A possible area of fog-based research is the investigation of the best methods for work relocation.
- The classification of VMs is crucial to managing the varying workloads of various user groups. In order to separate the VMs according to the specifications, an ensemble clustering technique is required. As a result, locally, under the least-configured (homogeneous) VMs, tasks with minimal bandwidth, MIPS, resource usage, and processing time can run. However, collaborative virtual machines (VMs), a combination of homogeneous and heterogeneous can handle complex computational operations.
- Cloud computing-like services can be offered using fog computing. In cloud computing, users are often billed based on their utilization on a horizontal scale. In contrast to cloud computing, fog vertical resource allocation significantly increases costs for both users and providers. As a result, compared to policies that are more cloud-oriented, the pricing and billing rules in fog often differ greatly. Additionally, because Fog-based services lack appropriate pricing and billing procedures, consumers frequently have trouble finding qualified vendors for executing SLA. A correct pricing and invoicing structure for fog-based services in such a situation will undoubtedly be viewed as a possible advancement in the fog computing sector.
- The development of a real-world testbed for assessing the effectiveness of fog-based policies is frequently costly and frequently not scalable. Therefore, many academics explore effective toolkits for fog modeling in order to do a preliminary evaluation of suggested fog computing systems. However, there aren't many fog simulators accessible right now. Future research could focus on creating a new, effective simulator for fog computing.
- In essence, fog computing was created to extend cloud-based services like PaaS, IaaS, SaaS, etc., to IoT devices and sensors nearby. Since fog is structured differently from the cloud, current standards and associated programming languages must be modified or improved in order to support cloud-based services in fog. Moreover, the creation of effective networking protocols and user interfaces is also required for the seamless and adaptable management of a high number of connections in fog.
- Non-uniform resource utilization in dynamic VM consolidation could be prevented by considering a number of factors, including bandwidth, memory, CPU cycle time, aim for computationally intensive tasks, present and future resource needs, latency, and memory. These components rely heavily on accurate resource forecasts. These issues might be resolved by applying artificial neural networks (ANNs) and some training data, such as historical resource utilization statistics.

- Recognizing the number of requests could help in figuring out how many resources need to be provisioned. Investigating the historical data could be used to train this restriction. Furthermore, several prospective ML/DL algorithms, like LSTM, XGBoost, etc., could be used to forecast the peak CPU consumption. A single ML/DL algorithm cannot handle time-series data and non-linear workloads. It could be enhanced by combining the ML/DL and DL techniques to provide a more reliable result.

To make the proposed research directions more practical and research-oriented, we now associate each of them with specific performance indicators that can be used to guide experimental design and evaluate success. For instance, advancements such as self-customizable and interoperable fog node architectures are expected to enhance scalability, resource adaptability, and QoS assurance in dynamic environments. Similarly, the adoption of ensemble learning models for workload prediction aims to minimize latency, optimize task distribution, and improve forecast accuracy under workload fluctuations.

The following Table 7 provides a classification matrix that maps each proposed direction to its potential contribution area and relevant quantifiable evaluation metrics, thereby offering researchers a clearer path for future experimentation and validation.

## IX. SUMMARY AND CONCLUSION

This study presents a comprehensive review of resource management and task offloading in fog computing, emphasizing the role of Machine Learning (ML) and Deep Learning (DL) in addressing the complexity of dynamic and heterogeneous environments. The proposed IoT–Edge–Fog–Cloud architecture outlines the functional layers and supports the integration of intelligent, data-driven decision-making for resource allocation. Through a comparative analysis of existing task offloading strategies and prior survey works, we highlight how ML/DL models enable adaptive, predictive, and efficient management of computing resources. The paper also identifies key open research issues, discusses simulation tools and frameworks, and frames critical research questions that pave the way for future exploration. This work aims to guide researchers in developing intelligent and scalable solutions for fog-based IoT systems, supporting the continued evolution of the fog paradigm in real-world applications.

## CONFLICT OF INTEREST

The authors declare there is no conflict of interest.

## REFERENCES


[1] M. Kaushik and S. Majhi, "A state-of-art on cloud load balancing algorithms," *Int. J. Comput. Digit. Syst.*, vol. 9, no. 2, pp. 201–220, Jan. 2020.

[2] S. S. Tripathy, K. Mishra, D. S. Roy, K. Yadav, A. Alferaidi, W. Viriyasitavat, J. Sharmila, G. Dhiman, and R. K. Barik, "State-of-the-art load balancing algorithms for mist-fog-cloud assisted paradigm: A review and future directions," *Arch. Comput. Methods Eng.*, vol. 30, no. 4, pp. 2725–2760, May 2023, doi: 10.1007/s11831-023-09885-1.

[3] M. Agarwal and G. M. S. Srivastava, "Cloud computing: A paradigm shift in the way of computing," *Int. J. Mod. Educ. Comput. Sci.*, vol. 9, no. 12, p. 38, 2017.

[4] A. Jyoti, M. Shrimali, and R. Mishra, "Cloud computing and load balancing in cloud computing—Survey," in *Proc. 9th Int. Conf. Cloud Comput., Data Sci. Eng. (Confluence)*, Jan. 2019, pp. 51–55.

[5] N. A. Angel, D. Ravindran, P. M. D. R. Vincent, K. Srinivasan, and Y.-C. Hu, "Recent advances in evolving computing paradigms: Cloud, edge, and fog technologies," *Sensors*, vol. 22, no. 1, p. 196, Dec. 2021.

[6] S. N. Srirama, "A decade of research in fog computing: Relevance, challenges, and future directions," 2023, *arXiv:2305.01974*.

[7] M. Fahimullah, S. Ahvar, and M. Trocan, "A review of resource management in fog computing: Machine learning perspective," 2022, *arXiv:2209.03066*.

[8] N. Kaur, A. Kumar, and R. Kumar, "A systematic review on task scheduling in fog computing: Taxonomy, tools, challenges, and future directions," *Concurrency Comput., Pract. Exper.*, vol. 33, no. 21, Nov. 2021, Art. no. e6432.

[9] T. Khan, W. Tian, G. Zhou, S. Ilager, M. Gong, and R. Buyya, "Machine learning (ML)-centric resource management in cloud computing: A review and future directions," *J. Netw. Comput. Appl.*, vol. 204, Aug. 2022, Art. no. 103405.

[10] J. P. Higgins, J. Thomas, J. Chandler, M. Cumpston, T. Li, M. J. Page, and V. A. Welch, *Cochrane Handbook for Systematic Reviews of Interventions*. Hoboken, NJ, USA: Wiley, 2019.

[11] L. Karthik, G. Kumar, T. Keswani, A. Bhattacharyya, S. S. Chandar, and K. V. B. Rao, "Protease inhibitors from marine actinobacteria as a potential source for antimalarial compound," *PLoS ONE*, vol. 9, no. 3, Mar. 2014, Art. no. e90972.

[12] M. Newman and D. Gough, "Systematic reviews in educational research: Methodology, perspectives and application," in *Systematic Reviews in Educational Research: Methodology, Perspectives and Application*. Wiesbaden, Germany: Springer, 2020, pp. 3–22. [Online]. Available: https://doi.org/10.1007/978-3-658-27602-7_1

[13] J. Schopfel and D. J. Farace, "Grey literature," in *Encyclopaedia of Library and Information Sciences*. Boca Raton, FL, USA: CRC Press, 2010, pp. 2029–2039.

[14] C. Bitsakos, I. Konstantinou, and N. Koziris, "DERP: A deep reinforcement learning cloud system for elastic resource provisioning," in *Proc. IEEE Int. Conf. Cloud Comput. Technol. Sci. (CloudCom)*, Dec. 2018, pp. 21–29.

[15] M. Cheng, J. Li, and S. Nazarian, "DRL-cloud: Deep reinforcement learning-based resource provisioning and task scheduling for cloud service providers," in *Proc. 23rd Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2018, pp. 129–134.

[16] D. Ding, X. Fan, Y. Zhao, K. Kang, Q. Yin, and J. Zeng, "Q-learning based dynamic task scheduling for energy-efficient cloud computing," *Future Gener. Comput. Syst.*, vol. 108, pp. 361–371, Jul. 2020.

[17] G. Sun, T. Zhan, B. Gordon Owusu, A.-M. Daniel, G. Liu, and W. Jiang, "Revised reinforcement learning based on anchor graph hashing for autonomous cell activation in cloud-RANs," *Future Gener. Comput. Syst.*, vol. 104, pp. 60–73, Mar. 2020.

[18] M. Goudarzi, M. Palaniswami, and R. Buyya, "Scheduling IoT applications in edge and fog computing environments: A taxonomy and future directions," *ACM Comput. Surv.*, vol. 55, no. 7, pp. 1–41, Jul. 2023.

[19] S. Ketu and P. K. Mishra, "Cloud, fog and mist computing in IoT: An indication of emerging opportunities," *IETE Tech. Rev.*, vol. 39, no. 3, pp. 713–724, May 2022.

[20] F. Saeik, "Task offloading in edge and cloud computing: A survey on mathematical, artificial intelligence and control theory solutions," *Comput. Netw.*, vol. 195, Aug. 2021, Art. no. 108177.

[21] D. A. Shafiq, N. Z. Jhanjhi, and A. Abdullah, "Load balancing techniques in cloud computing environment: A review," *J. King Saud Univ.-Comput. Inf. Sci.*, vol. 34, no. 7, pp. 3910–3933, 2022.

[22] A. A. Alli and M. M. Alam, "The fog cloud of things: A survey on concepts, architecture, standards, tools, and applications," *Internet Things*, vol. 9, Mar. 2020, Art. no. 100177.

[23] M. S. Aslanpour, S. S. Gill, and A. N. Toosi, "Performance evaluation metrics for cloud, fog and edge computing: A review, taxonomy, benchmarks and standards for future research," *Internet Things*, vol. 12, Dec. 2020, Art. no. 100273.

[24] J. Moura and D. Hutchison, "Fog computing systems: State of the art, research issues and future trends, with a focus on resilience," *J. Netw. Comput. Appl.*, vol. 169, Nov. 2020, Art. no. 102784.
[25] K. Yunana, A. A. Alfa, S. Misra, R. Damasevicius, R. Maskeliunas, and J. Oluranti, "Internet of Things: Applications, adoptions and components—A conceptual overview," in *Proc. 20th Int. Conf. Hybrid Intell. Syst. Hybrid Intell. Syst. (HIS)*, Dec. 2020, pp. 494–504.
[26] K. Cao, Y. Liu, G. Meng, and Q. Sun, "An overview on edge computing research," *IEEE Access*, vol. 8, pp. 85714–85728, 2020.
[27] W. Z. Khan, E. Ahmed, S. Hakak, I. Yaqoob, and A. Ahmed, "Edge computing: A survey, future generation computer systems," *Future Generation Comput. Syst.*, vol. 97, pp. 219–235, Aug. 2019. [Online]. Available: https://doi.org/10.1016/j.future.2019.02.050
[28] Y. Liu, J. E. Fieldsend, and G. Min, "A framework of fog computing: Architecture, challenges, and optimization," *IEEE Access*, vol. 5, pp. 25445–25454, 2017.
[29] K. H. Abdulkareem, M. A. Mohammed, S. S. Gunasekaran, M. N. Al-Mhiqani, A. A. Mutlag, S. A. Mostafa, N. S. Ali, and D. A. Ibrahim, "A review of fog computing and machine learning: Concepts, applications, challenges, and open issues," *IEEE Access*, vol. 7, pp. 153123–153140, 2019.
[30] A. Yousefpour, C. Fung, T. Nguyen, K. Kadiyala, F. Jalali, A. Niakanlahiji, J. Kong, and J. P. Jue, "All one needs to know about fog computing and related edge computing paradigms: A complete survey," *J. Syst. Archit.*, vol. 98, pp. 289–330, Sep. 2019.
[31] H. Bangui, S. Rakrak, S. Raghay, and B. Buhnova, "Moving to the edge-cloud-of-things: Recent advances and future research directions," *Electronics*, vol. 7, no. 11, p. 309, Nov. 2018.
[32] H. Atlam, R. Walters, and G. Wills, "Fog computing and the Internet of Things: A review," *Big Data Cognit. Comput.*, vol. 2, no. 2, p. 10, Apr. 2018.
[33] H. Elazhary, "Internet of Things (IoT), mobile cloud, cloudlet, mobile IoT, IoT cloud, fog, mobile edge, and edge emerging computing paradigms: Disambiguation and research directions," *J. Netw. Comput. Appl.*, vol. 128, pp. 105–140, Feb. 2019.
[34] M. Mukherjee, L. Shu, and D. Wang, "Survey of fog computing: Fundamental, network applications, and research challenges," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 3, pp. 1826–1857, 3rd Quart., 2018.
[35] C. Mouradian, D. Naboulsi, S. Yangui, R. H. Glitho, M. J. Morrow, and P. A. Polakos, "A comprehensive survey on fog computing: State-of-the-art and research challenges," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 1, pp. 416–464, 1st Quart., 2018.
[36] B. Yong, W. Wei, K.-C. Li, J. Shen, Q. Zhou, M. Wozniak, D. Połap, and R. Damaševičius, "Ensemble machine learning approaches for webshell detection in Internet of Things environments," *Trans. Emerg. Telecommun. Technol.*, vol. 33, no. 6, Jun. 2022, Art. no. e4085.
[37] P. Mach and Z. Becvar, "Mobile edge computing: A survey on architecture and computation offloading," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 3, pp. 1628–1656, 3rd Quart., 2017.
[38] J. Lin, W. Yu, N. Zhang, X. Yang, H. Zhang, and W. Zhao, "A survey on Internet of Things: Architecture, enabling technologies, security and privacy, and applications," *IEEE Internet Things J.*, vol. 4, no. 5, pp. 1125–1142, Oct. 2017.
[39] R. Mahmud, R. Kotagiri, and R. Buyya, "Fog computing: A taxonomy, survey and future directions," in *Internet of Everything: Algorithms, Methodologies, Technologies and Perspectives*. Singapore: Springer, 2018, pp. 103–130. [Online]. Available: https://doi.org/10.1007/978-981-10-5861-5_5
[40] P. Hu, S. Dhelim, H. Ning, and T. Qiu, "Survey on fog computing: Architecture, key technologies, applications and open issues," *J. Netw. Comput. Appl.*, vol. 98, pp. 27–42, Nov. 2017.
[41] L. Atzori, A. Iera, and G. Morabito, "Understanding the Internet of Things: Definition, potentials, and societal role of a fast evolving paradigm," *Ad Hoc Netw.*, vol. 56, pp. 122–140, Mar. 2017.
[42] S. Katoch and J. Thakur, "Load balancing algoritms in cloud computing environment: A review," *Int. J. Recent Innov. Trends Comput. Commun.*, vol. 2, no. 8, pp. 2151–2156, 2014.
[43] S. K. Mishra, B. Sahoo, and P. P. Parida, "Load balancing in cloud computing: A big picture," *J. King Saud Univ. Comput. Inf. Sci.*, vol. 32, no. 2, pp. 149–158, Feb. 2020.
[44] P. Kumar and R. Kumar, "Issues and challenges of load balancing techniques in cloud computing: A survey," *ACM Comput. Surveys*, vol. 51, no. 6, pp. 1–35, Nov. 2019.
[45] K. Lakhwani, "An extensive survey on load balancing techniques in cloud computinG," *J. Gujarat Res. Soc.*, vol. 21, no. 10s, pp. 309–319, 2019.
[46] S. Afzal and G. Kavitha, "Load balancing in cloud computing—A hierarchical taxonomical classification," *J. Cloud Comput.*, vol. 8, no. 1, pp. 1–24, Dec. 2019.
[47] A. Hota, S. Mohapatra, and S. Mohanty, "Survey of different load balancing approach-based algorithms in cloud computing: A comprehensive review," in *Proc. Int. Conf. Comput. Intell. Data Mining (CIDM)*, Singapore: Springer, 2019, pp. 99–110.
[48] P. R. Kathalkar and A. V. Deorankar, "A review on different load balancing algorithm in cloud computing," *Int. Res. J. Eng. Technol.*, vol. 5, no. 2, pp. 1–3, 2018.
[49] D. S. Kumar, "A literature review on load balancing mechanisms in cloud computing," *Int. J. Adv. Res. Comput. Sci.*, vol. 9, no. 1, pp. 432–435, Feb. 2018.
[50] M. Kaur and D. B. Verma, "A review on various load balancing algorithms with Merits–Demerits in cloud computing," *Int. J. Adv. Eng. Res. Dev.*, vol. 5, no. 5, p. 1, 2018.
[51] S. Singh, "Load balancing algorithms in cloud computing environment," *Int. J. Adv. Res. Comput. Sci.*, vol. 9, no. 2, p. 397, 2018.
[52] S. Hamadah, "A survey: A comprehensive study of static, dynamic and hybrid load balancing algorithms," *Int. J. Comput. Sci. Inf. Technol. Secur. (IJCSITS)*, vol. 7, no. 2, pp. 2249–9555, Mar. 2017.
[53] S. S. and S. G., "Comparative analysis and evaluation of load balancing algorithms," *Int. J. Comput. Appl.*, vol. 171, no. 5, pp. 6–11, Aug. 2017.
[54] M. Archana and M. Shastry, "A review paper on various load balancing algorithms in cloud computing," *J. Eng. Appl. Sci.*, vol. 12, no. 9, pp. 8579–8585, 2017.
[55] S. Gupta, A. Dixit, and H. Dev, "A study on various load balancing algorithms for response time reduction in cloud," *Int. J. Current Eng. Sci. Res.*, vol. 4, no. 10, p. 1, 2017.
[56] A. Thakur and M. S. Goraya, "A taxonomic survey on load balancing in cloud," *J. Netw. Comput. Appl.*, vol. 98, pp. 43–57, Nov. 2017.
[57] M. Alam and Z. A. Khan, "Issues and challenges of load balancing algorithm in cloud computing environment," *Indian J. Sci. Technol.*, vol. 10, no. 25, pp. 1–12, 2017.
[58] S. Joshi and U. Kumari, "A comprehensive analysis of existing load balancing algorithms in cloud network," *Mody Univ. Int. J. Comput. Eng. Res.*, vol. 1, no. 2, pp. 71–75, 2017.
[59] A. B. Singh, S. Bhat, R. Raju, and R. D'Souza, "Survey on various load balancing techniques in cloud computing," *Adv. Comput.*, vol. 7, no. 2, pp. 28–34, 2017.
[60] J. Ghomi, "Load-balancing algorithms in cloud computing: A survey," *J. Netw. Comput. Appl.*, vol. 88, pp. 50–71, Jun. 2017.
[61] A. S. Milani and N. J. Navimipour, "Load balancing mechanisms and techniques in the cloud environments: Systematic literature review and future trends," *J. Netw. Comput. Appl.*, vol. 71, pp. 86–98, Aug. 2016.
[62] Z. M. Elngomi and K. Khanfar, "A comparative study of load balancing algorithms: A review paper," *Int. J. Comput. Sci. Mobile Comput.*, vol. 5, no. 6, pp. 448–458, 2016.
[63] D. Gabi, A. Samad, and A. Zainal, "Systematic review on existing load balancing techniques in cloud computing," *Int. J. Comput. Appl.*, vol. 125, no. 9, pp. 16–24, Sep. 2015.
[64] K. Karthika, K. Kanakambal, and R. Balasubramaniam, "Load balancing algorithm review's in cloud environment," *IJERGS*, vol. 3, no. 3, pp. 661–667, 2015.
[65] S. Kapoor, "A survey on dynamic load balancing algorithms in cloud computing," *Adv. Comput. Sci. Inf. Technol.*, vol. 2, no. 7, pp. 87–91, 2015.
[66] H. Mj, J. P. Martin, Y. Sastri, and A. Babu, "A review on load balancing algorithms in cloud," *Comput. Technol. Appl.*, vol. 5, no. 2, pp. 640–645, 2014.
[67] Y. Jiang, "A survey of task allocation and load balancing in distributed systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 27, no. 2, pp. 585–599, Feb. 2016.
[68] R. Buyya, C. S. Yeo, and S. Venugopal, "Market-oriented cloud computing: Vision, hype, and reality for delivering it services as computing utilities," in *Proc. 10th IEEE Int. Conf. High Perform. Comput. Commun.*, Sep. 2008, pp. 5–13.
[69] Q. T. Minh, D. T. Nguyen, A. L. Van Le, H. D. Nguyen, and A. Truong, "Toward service placement on fog computing landscape," in *Proc. 4th NAFOSTED Conf. Inf. Comput. Sci.*, Nov. 2017, pp. 291–296.

[70] Z. Tong, H. Chen, X. Deng, K. Li, and K. Li, "A scheduling scheme in the cloud computing environment using deep Q-learning," *Inf. Sci.*, vol. 512, pp. 1170–1191, Feb. 2020.

[71] R. Singhal and A. Singhal, "A feedback-based combinatorial fair economical double auction resource allocation model for cloud computing," *Future Generat. Comput. Syst.*, vol. 115, pp. 780–797, Feb. 2021.

[72] T. Dong, F. Xue, C. Xiao, and J. Li, "Task scheduling based on deep reinforcement learning in a cloud manufacturing environment," *Concurrency Comput., Pract. Exper.*, vol. 32, no. 11, Jun. 2020, Art. no. e5654.

[73] H. Lu, C. Gu, F. Luo, W. Ding, and X. Liu, "Optimization of lightweight task offloading strategy for mobile edge computing based on deep reinforcement learning," *Future Gener. Comput. Syst.*, vol. 102, pp. 847–861, Jan. 2020.

[74] Z. Cao, P. Zhou, R. Li, S. Huang, and D. Wu, "Multiagent deep reinforcement learning for joint multichannel access and task offloading of mobile-edge computing in Industry 4.0," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6201–6213, Jul. 2020.

[75] M. Li, F. R. Yu, P. Si, W. Wu, and Y. Zhang, "Resource optimization for delay-tolerant data in blockchain-enabled IoT with edge computing: A deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9399–9412, Oct. 2020.

[76] N. Shan, X. Cui, and Z. Gao, "'DRL+ FL': An intelligent resource allocation model based on deep reinforcement learning for Mobile Edge Computing," *Comput. Commun.*, vol. 160, pp. 14–24, Jul. 2020.

[77] J. Wang, J. Hu, G. Min, A. Y. Zomaya, and N. Georgalas, "Fast adaptive task offloading in edge computing based on meta reinforcement learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 1, pp. 242–253, Jan. 2021.

[78] W. Guo, W. Tian, Y. Ye, L. Xu, and K. Wu, "Cloud resource scheduling with deep reinforcement learning and imitation learning," *IEEE Internet Things J.*, vol. 8, no. 5, pp. 3576–3586, Mar. 2021.

[79] S. Kardani-Moghaddam, R. Buyya, and K. Ramamohanarao, "ADRL: A hybrid anomaly-aware deep reinforcement learning-based resource scaling in clouds," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 3, pp. 514–526, Mar. 2021.

[80] E. M. Dogo, A. F. Salami, C. O. Aigbavboa, and T. Nkonyana, "Taking cloud computing to the extreme edge: A review of mist computing for smart cities and Industry 4.0 in Africa," in *Edge Computing: From Hype to Reality*. Cham, Switzerland: Springer, 2019, pp. 107–132. [Online]. Available: https://doi.org/10.1007/978-3-319-99061-3_7

[81] M. Liyanage, C. Chang, and S. N. Srirama, "mePaaS: Mobile-embedded platform as a service for distributing fog computing to edge nodes," in *Proc. 17th Int. Conf. Parallel Distrib. Comput., Appl. Technol. (PDCAT)*, Dec. 2016, pp. 73–80.

[82] J. S. Preden, K. Tammemäe, A. Jantsch, M. Leier, A. Riid, and E. Calis, "The benefits of self-awareness and attention in fog and mist computing," *Computer*, vol. 48, no. 7, pp. 37–45, Jul. 2015.

[83] J. S. Preden, "Evolution of mist computing from fog and cloud computing," THIINECT White Paper, pp. 1–4, Aug. 2016.

[84] L. Liu, D. Qi, N. Zhou, and Y. Wu, "A task scheduling algorithm based on classification mining in fog computing environment," *Wireless Commun. Mobile Comput.*, vol. 2018, no. 1, Jan. 2018, Art. no. 2102348.

[85] S. Sharma and H. Saini, "A novel four-tier architecture for delay aware scheduling and load balancing in fog environment," *Sustain. Comput., Informat. Syst.*, vol. 24, Dec. 2019, Art. no. 100355.

[86] P. Gazori, D. Rahbari, and M. Nickray, "Saving time and cost on the scheduling of fog-based IoT applications using deep reinforcement learning approach," *Future Gener. Comput. Syst.*, vol. 110, pp. 1098–1115, Sep. 2020.

[87] M. M. Razaq, S. Rahim, B. Tak, and L. Peng, "Fragmented task scheduling for load-balanced fog computing based on Q-learning," *Wireless Commun. Mobile Comput.*, vol. 2022, pp. 1–9, Mar. 2022.

[88] C. Chakraborty, K. Mishra, S. K. Majhi, and H. K. Bhuyan, "Intelligent latency-aware tasks prioritization and offloading strategy in distributed fog-cloud of things," *IEEE Trans. Ind. Informat.*, vol. 19, no. 2, pp. 2099–2106, Feb. 2023.

[89] K. Mishra, G. N. V. Rajareddy, U. Ghugar, G. S. Chhabra, and A. H. Gandomi, "A collaborative computation and offloading for compute-intensive and latency-sensitive dependency-aware tasks in dew-enabled vehicular fog computing: A federated deep Q-Learning approach," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 4, pp. 4600–4614, Dec. 2023.

[90] (2023). *Cisco. Cisco IOx*. Accessed: Mar. 13, 2023. [Online]. Available: https://www.cisco.com/c/en/us/products/cloud-systems-management/iox/index.htML/DL

[91] S. Tuli, R. Mahmud, S. Tuli, and R. Buyya, "FogBus: A blockchain-based lightweight framework for edge and fog computing," *J. Syst. Softw.*, vol. 154, pp. 22–36, Aug. 2019.

[92] C. Chang, S. Narayana Srirama, and R. Buyya, "Indie fog: An efficient fog-computing infrastructure for the Internet of Things," *Computer*, vol. 50, no. 9, pp. 92–98, 2017.

[93] H. Gupta, A. Vahid Dastjerdi, S. K. Ghosh, and R. Buyya, "IFogSim: A toolkit for modeling and simulation of resource management techniques in the Internet of Things, edge and fog computing environments," *Softw., Pract. Exper.*, vol. 47, no. 9, pp. 1275–1296, Sep. 2017.

[94] C. Puliafito, D. M. Gonçalves, M. M. Lopes, L. L. Martins, E. Madeira, E. Mingozzi, O. Rana, and L. F. Bittencourt, "MobFogSim: Simulation of mobility and migration for fog computing," *Simul. Model. Pract. Theory*, vol. 101, May 2020, Art. no. 102062.

[95] C. Sonmez, A. Ozgovde, and C. Ersoy, "EdgeCloudSim: An environment for performance evaluation of edge computing systems," *Trans. Emerg. Telecommun. Technol.*, vol. 29, no. 11, Nov. 2018, Art. no. e3493.

[96] M. Dumas, M. La Rosa, J. Mendling, and H. A. Reijers, *Fundamentals of Business Process Management*. Cham, Germany: Springer, 2013.

[97] R. Morabito, I. Farris, A. Iera, and T. Taleb, "Evaluating performance of containerized IoT services for clustered devices at the network edge," *IEEE Internet Things J.*, vol. 4, no. 4, pp. 1019–1030, Aug. 2017.

[98] S. N. Srirama and D. Vemuri, "CANTO: An actor model-based distributed fog framework supporting neural networks training in IoT applications," *Comput. Commun.*, vol. 199, pp. 1–9, Feb. 2023.

[99] (2023). *Apache Software Foundation. Apache Edgent*. Accessed: Mar. 13, 2023. [Online]. Available: https://incubator.apache.org/projects/edgent.htML/DL

[100] A. Mahapatra, K. Mishra, and R. Pradhan, "Next generation task offloading techniques in evolving computing paradigms: Comparative analysis, current challenges, and future research perspectives," *Arch. Comput. Methods Eng.*, vol. 31, no. 3, pp. 1405–1474, 2023, doi: 10.1007/s11831-023-10021-2.

[101] A. Mahapatra, S. K. Majhi, K. Mishra, R. Pradhan, D. C. Rao, and S. K. Panda, "An energy-aware task offloading and load balancing for latency-sensitive IoT applications in the fog-cloud continuum," *IEEE Access*, vol. 12, pp. 14334–14349, 2024, doi: 10.1109/ACCESS.2024.3357122.

[102] S. F. Ahmed, M. S. B. Alam, and M. Hassan, "Deep learning modelling techniques: Current progress, applications, advantages, and challenges," *Artif. Intell. Rev.*, vol. 56, no. 11, pp. 13521–13617, 2023, doi: 10.1007/s10462-023-10466-8.

[103] D. Yazdani, M. N. Omidvar, D. Yazdani, J. Branke, T. T. Nguyen, A. H. Gandomi, Y. Jin, and X. Yao, "Robust optimization over time: A critical review," *IEEE Trans. Evol. Comput.*, vol. 28, no. 5, pp. 1265–1285, Oct. 2024.

[104] S. F. Ahmed, S. Shuravi, S. Afrin, S. J. Rafa, M. Hoque, and A. H. Gandomi, "The power of Internet of Things (IoT): Connecting the dots with cloud, edge, and fog computing," 2023, *arXiv:2309.03420*.

[105] L. Gong, H. Chen, Y. Chen, T. Yao, C. Li, S. Zhao, and G. Han, "DPNet: Dynamic pooling network for accurate and efficient size-aware tiny object detection," *IEEE Internet Things J.*, vol. 12, no. 14, pp. 26387–26400, Jul. 2025.



**GURPREET SINGH CHHABRA** received the Ph.D. degree in CSE. He has 15 years of teaching experience. He is currently an Assistant Professor with the Computer Science and Engineering Department, GITAM School of Technology, GITAM Deemed to be University, Visakhapatnam. Also, he has credit for many international articles, patents, book chapters, and books. His qualifications are fortified with a great deal of creativity and problem-solving skills. His research interests include deep learning, machine learning, data science, and fog computing. He is a Life Member of ISTE and IAENG.



**GOLUGURI N. V. RAJAREDDY** received the Ph.D. degree from NIT Silchar, in 2023. He is currently an Assistant Professor with the Department of Computer Science and Engineering, School of Technology, Gandhi Institute of Technology and Management (GITAM), GITAM Deemed to be University, Visakhapatnam, Andhra Pradesh, India. He has eight years of full-time teaching experience. He has published five research articles in various reputed international journals. His research interests include image processing and deep learning. He is a member of IAENG, UACEE, and IFERP.



**ABHIJEET MAHAPATRA** received the B.Tech. degree in computer science and engineering (CSE) from BPUT, Rourkela, Odisha, India, and the master's and Ph.D. degrees in CSE from the Veer Surendra Sai University of Technology (VSSUT), Burla, Odisha, India. He is currently an Assistant Professor with the School of Computer Engineering (SoCE), Manipal Institute of Technology Bengaluru, Manipal Academy of Higher Education. He has published seven journal articles and three conference papers. His research interests include task scheduling and load balancing with the IoT resource management in cloud and fog computing environments.



**S. SUDHEER MANGALAMPALLI** (Senior Member, IEEE) is currently an Associate Professor with the School of Computer Engineering (SoCE), Manipal Institute of Technology Bengaluru, Manipal Academy of Higher Education (MAHE). He is also an Assistant Director of Research at MAHE, Bengaluru Campus, where he is looking after all the research activities of both faculty and students. He is also a Passionate Researcher. He has 48 publications, which are indexed in Scopus and SCI databases. His research interests include cloud computing, edge computing, fog computing, and machine learning. He is a reviewer for various SCI-indexed journals.



**KSHIRA SAGAR SAHOO** (Senior Member, IEEE) received the M.Tech. degree from Indian Institute of Technology Kharagpur, India, in 2014, and the Ph.D. degree from NIT Rourkela, India, in 2019. He was a Postdoctoral Researcher (a Kempe Fellow) at Umeå University, Umeå, Sweden, from October 2022 to September 2024. Currently, he is working as an Assistant Professor with Department of CSE, SRM University-AP, India. He has published more than 90 research papers in international journals and conferences, including IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT, IEEE SYSTEMS JOURNAL, IEEE INTERNET OF THINGS JOURNAL, ACM TOMM, ACM TIOT, and MOBICOM. His research interests include SDN, edge computing, distributed computing, and urban anomaly detection.



**DEEPAK SETHI** (Member, IEEE) received the Ph.D. degree, in 2018. He is currently an Associate Professor with the School of Computer Engineering (SoCE), Manipal Institute of Technology Bengaluru, Manipal Academy of Higher Education, Manipal, India. He has published around 30 papers in various reputed journals and conferences. His research interests include machine learning, fog computing, and the Internet of Medical Things. He has served as a Reviewer for many reputed journals, such as *Wireless Networks* and *The Journal of Supercomputing*.



**KAUSHIK MISHRA** (Senior Member, IEEE) received the Ph.D. degree in computer science and engineering from the Veer Surendra Sai University of Technology (VSSUT), Burla, in 2021. He is currently an Associate Professor with the School of Computer Engineering (SoCE), Manipal Institute of Technology Bengaluru, Manipal Academy of Higher Education, Manipal, India. His research interests include task consolidation, load balancing, privacy protection using security mechanisms for cloud computing, fog computing, and the Internet of Medical Things. His research findings are published in many journals of repute, such as IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS, IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS, IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT, *ACM Transactions on Internet Technology*, *Journal of Supercomputing*, *Journal of King Saud University*, and *Open Computer Science*; and conferences, like IEEE TENSYMP. He has bagged two best paper awards in two conferences. He has served as a Reviewer for many reputed journals, such as IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS, IEEE INTERNET OF THINGS JOURNAL, *Future Generation Computer Systems*, *Cluster Computing*, *The Journal of Supercomputing*, *Multimedia Tools and Applications*, *Journal of Grid Computing*, and IGI Global.